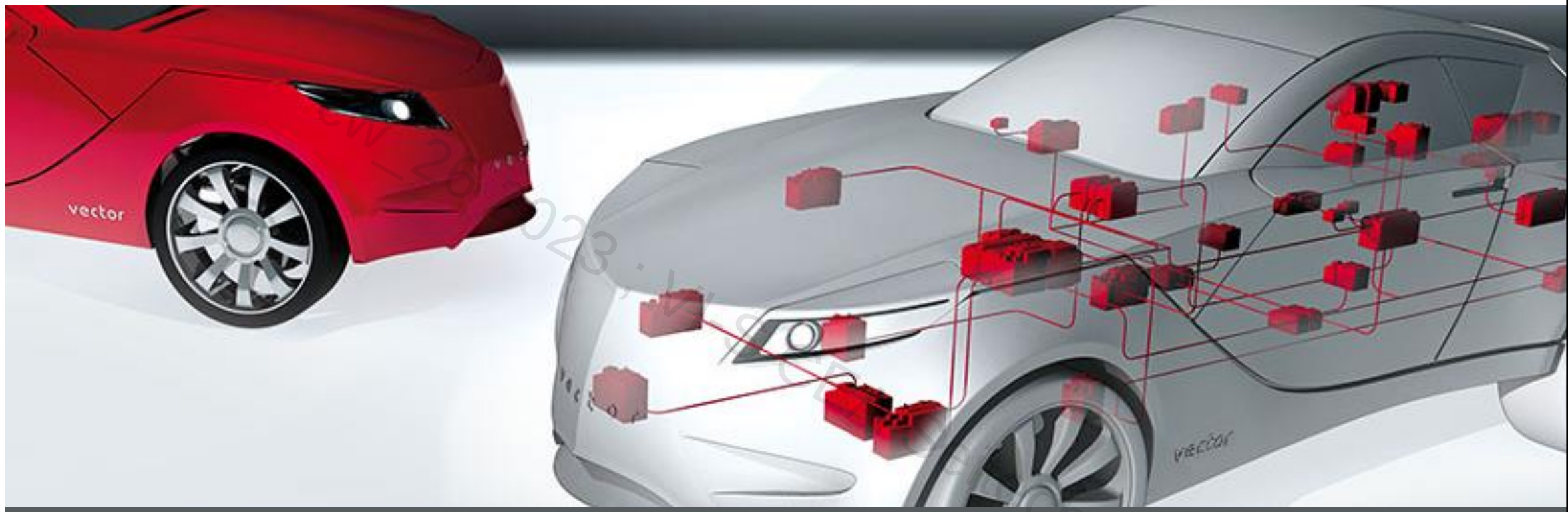
VECTOR



AUTOSAR in Practice

Memory Abstraction

Agenda

- **Principles of Memory Technologies**
- The MICROSAR.MEM solution
- Memory Handling
- Configuration
- NvBlockSwComponents
- Service Mapping
- Coming up next

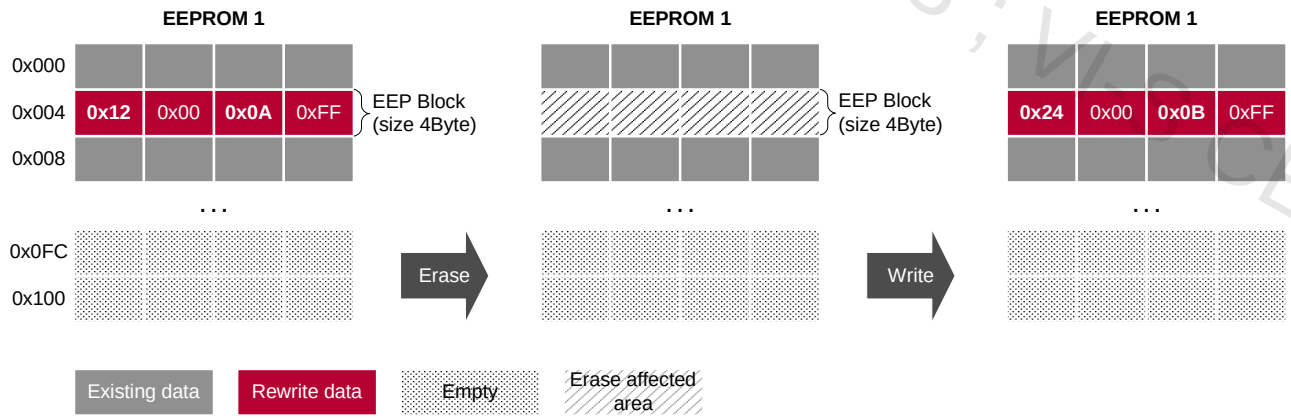
CW_26 2023 ; VI-S CEA Seitech

Differences between EEPROM and Flash

EEPROM

- Erasable and rewriteable in small blocks (1-4 bytes)
- Data can be rewritten to same memory cell easily

CW_26 2023 ; VL-CEA Seitech

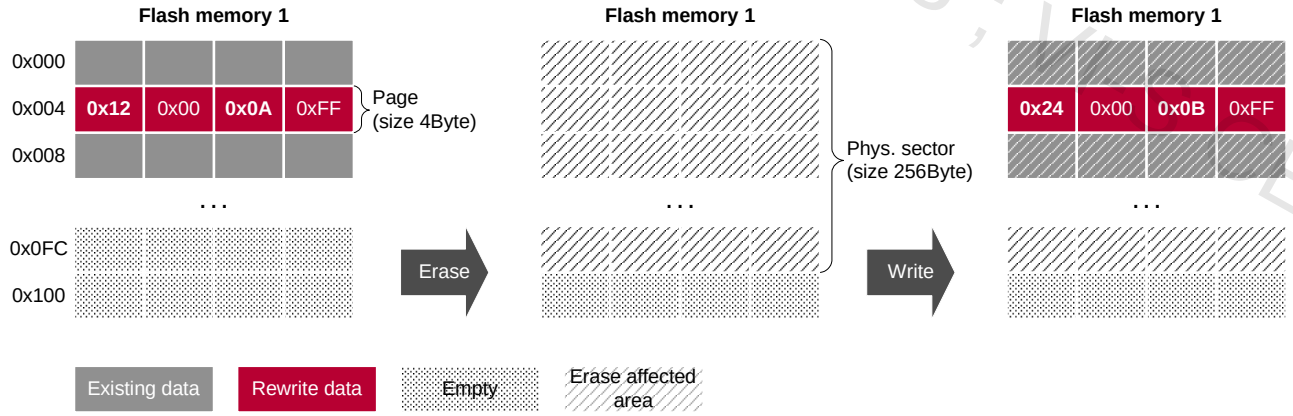


Differences between EEPROM and Flash

Flash EEPROM

- Has larger memory elements
- Smallest addressable memory unit (page) is not erasable on its own
- Multiple pages are combined into a physical sector which is erasable

cw_26_2023 ; VW EEA Seitech



Having different physical sector sizes also implies different time durations needed for sector erase. To erase 16KB may take hundreds of milliseconds, therefore an erase operation can last many seconds.

Agenda

Principles of Memory Technologies

- **The MICROSAR.MEM solution**

Memory Handling

Configuration

NvBlockSwComponents

Service Mapping

Coming up next

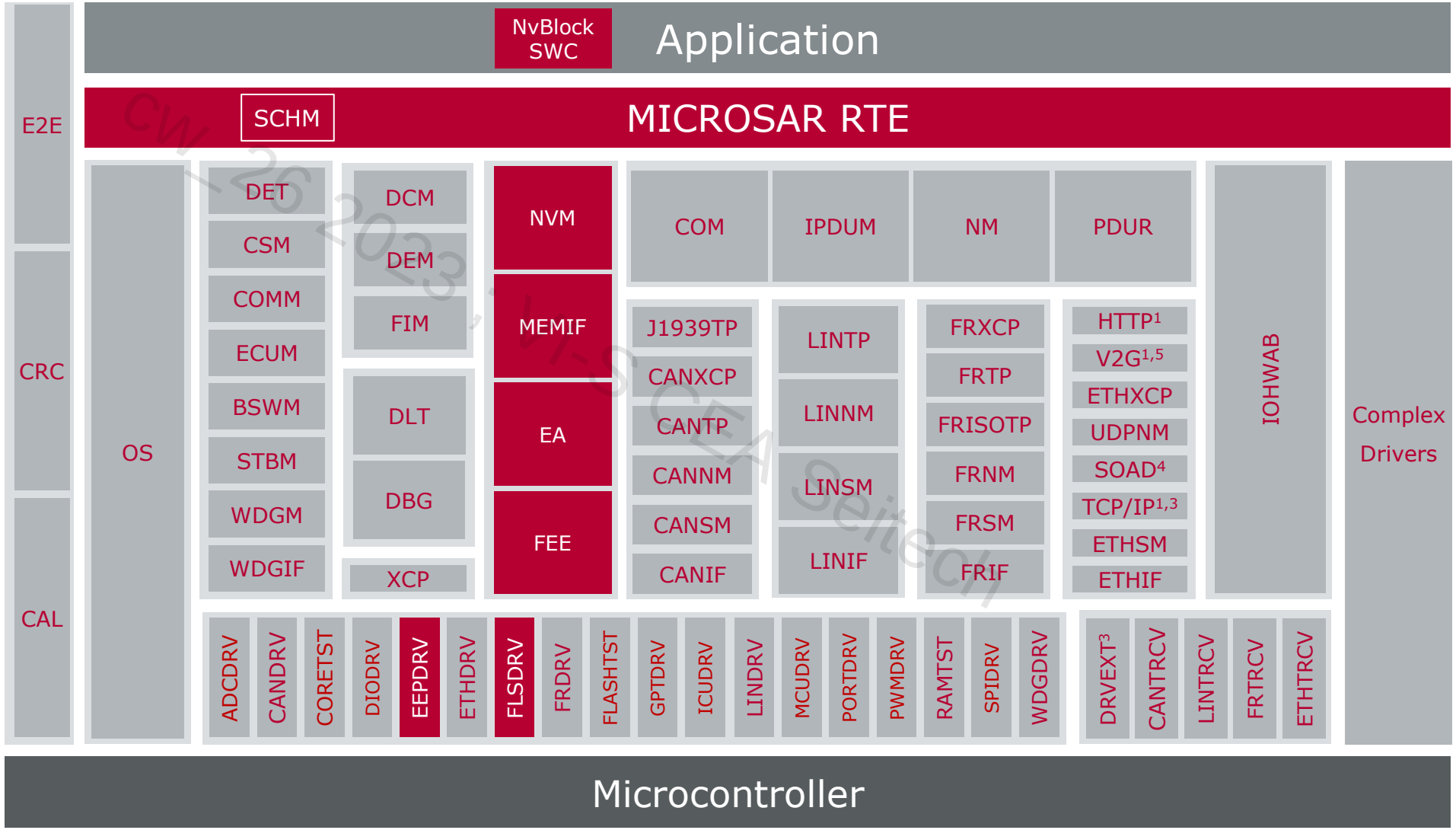
cw_26 2023 ; VI-S CEA Seitech

AUTOSAR BSW Modules for Memory Abstraction

- NVM
- MEMIF
- EA
- FEE
- EEPDRV
- FLSDRV

Optionally at RTE level:

- NvBlock-Sw-Component-Type



CW 26 2023 - V-S CEA Seitech

AUTOSAR BSW Modules for Memory Abstraction

NVM

- Provides abstract data storage
- Startup and shutdown handling for NV data
- Associates NV data with ROM defaults and RAM working copy
  - permanently or temporarily
- Data may be spread across multiple “devices”
- Explicit Synchronization support
- Improved Job handling
- BSWM support*
  - Block Status Mode Notification
    - Multi-Block, Single Block

*) BSWM Multi Block Job Status or Single Block Status Information: The NVM calls
- BswM_NvM_CurrentJobMode() (Multi Block, no call of the Multi Block Callback)
- BswM_NvM_CurrentBlockMode() (Single Block)
to notify the BSWM upon any job status changes of the Multi Block or Single Block respectively.

MEMIF

- Dispatches between memory technologies (FLASH or EEPROM)

FEE/EA

- Abstraction from different NV memory technologies
  (e.g. segmentation and number of write/erase cycles)

FLS/EEP

- NV memory hardware drivers
- Can access external devices e.g. via SPI

CW 26 2023 ; VI-SCOA Seitech

Nonvolatile Memory Manager (NVM)

- Queued job processing
  - Selectable between prioritized Queue or simple FIFO
  - Configurable queue depth
  - Separate queue for writing immediate data (e.g., crash data)
    - Guarantees job acceptance and fast start of job
  - Job cancellation API to abort pending requests
- Startup and shutdown handling
  - Read data blocks from NV at startup
  - Write data blocks at shutdown
  - Blocks are user-selectable
- Support of multiple RAM data blocks
  - Via configurable permanently-assigned and temporary RAM blocks
  - Temporary RAM Blocks allow SWCs to implement double-buffering

CN_26_2023 ; VI-S CEA Seitech

Nonvolatile Memory Manager (NVM)

- Data CRC
  - Configurable per-block: OFF, CRC16, CRC32
  - To protect data in NV memory – detection of errors
  - To protect data in RAM – consistency check after reset
    - E.g., due to watchdog reset or brown-out
  - Block-wise CRC recalculation in background (queued) to reduce CPU load
- Automatic / user-requested load of default data in case of read errors
  - From ROM or via callback (configurable per block)
- Configuration check at start-up - “dynamic configuration handling”
  - Ensures that current configuration is compatible with data layout of NV memory
  - Using NV data or block defaults in case of incompatibility
  - Potential incompatibilities are ignored if the “resistant against changed software” feature (“Res”) is enabled

CW_26_2023_VFS CFX Seitech

Nonvolatile Memory Manager (NVM)

- Write protection of data
  - Temporary (set/unset): configurable at runtime
  - Write Once: persistent
- Handling of RAM block states
  - Valid/Invalid data → avoid writing of invalid data
  - Modified/Unchanged data → avoid writing of unchanged data
    - `NvM_SetRamBlockStatus()` API exists to mark RAM data as
      - "Valid and Changed" OR
      - "Invalid and Unchanged"
- Redundancy
  - Increases data availability
    - In case of write error or abort (e.g. through reset), a second instance is available
  - Redundancy shall be used for data which is critical for application
  - Redundant blocks shall always be secured with a CRC

cw_26_2022, VI-S CEA Scitech

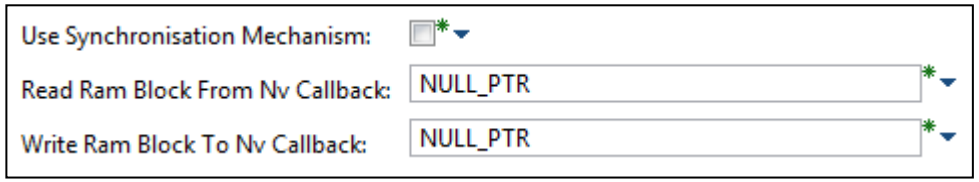
Nonvolatile Memory Manager (NVM)

- An NVRAM block consists of:
  - A RAM block (permanent/temporary) to transfer data between application and NVM.
    - Must be provided by application
  - RAM block management information (state, last request result)
  - One or more NV memory blocks of equal size
    - Payload size equal to RAM block size
    - Located in the same NV memory “device”
  - An optional default block which is either
    - A user-provided block of data in ROM (no CRC)
    - A user-provided callback

CW_26_2023 ; VI-S CEA Seitech

NVM Explicit Synchronization feature

- In the time between `Nvm_WriteBlock/NvM_ReadBlock` and `NvM_JobEndNotification`, the RAM working copy is not accessible or valid for the application
- Solution alternatives
  1. Double buffering using temporary RAM blocks (RAM mirror and double buffer in RTE/SWC)
  2. Explicit synchronization (double buffer as extra RAM mirror implemented by the NVM)
     - Can be enabled for single NvBlocks
       - Parameter: `NvMBlockUseSyncMechanism=TRUE|FALSE`
     - When the NVM reads or writes the RAM mirror, it calls a callback function configured in Configurator to collect data from or deliver data to the application RAM mirror
       > `NvMReadRamBlockFromNvM`
       > `NvMWriteRamBlockToNvM`



CW_26_2025, VI-S CEA Seitech

NVM Explicit Synchronization feature

NVM supports an optional Explicit Synchronization mechanism between application and NVM. It is implemented by an additional RAM mirror in the NVM module itself. The data is transferred by callbacks to the application which are called by the NVM module when needed. The synchronization mechanism can be configured for every NVRAM block separately. The NVM will create its RAM mirror based on the largest NVM blocks which uses explicit synchronization.

You can configure the names of the callback functions in Configurator:

Std_ReturnType <NvM_WriteRamBlockToNvm> (void* NvMBuffer)

Std_ReturnType <NvM_ReadRamBlockFromNvm> (const void* NvMBuffer)

Please note that for these callbacks are not Operations inside a Service Port Prototype. As a consequence, you have to provide *.c and *.h files where you will implement these functions. You have to include the *.h file as an “additional include” for the NVM module. Please also note that you have to remove the permanent RAM block from the respective block configuration, because the callbacks will copy the data without using the permanent block.

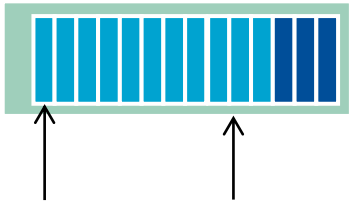
CW 26 2023, VI-SCEA Seitech

Flash EEPROM Emulation (FEE)

FEE is implemented according to the properties of Flash Memory

- Access to the memory using Flash pages only
  - These consist of several bytes
- A Flash page
  - Can be written once, typically*
  - Cannot be erased individually
- Sectors comprise several pages
  - Only sectors can be erased individually
    - All constituent pages are erased together
  - Limited number of write/erase cycles per cell
    - High utilization of a sector is advisable!

→ Therefore a **dynamic data layout** is used to access the Flash memory and exploit one sector as effectively as possible



*) The flash technology only allows to modify bits from 1 → 0, but not from 0 → 1 Therefore a flash page already written has to be erased prior to further usage.

cw_26 2023 ; VI-S CEA Seitech

Flash EEPROM Emulation (FEE)

- Uses Flash drivers (FLS) to access flash hardware
- Dedicated Data Flash or Read-While-Write feature required in hardware
- Manages data through “linked lists” within a FEE sector
- Memory is always allocated for the configured “chunk” size
- Automatic switching between two FEE sectors
  - Copies most recent data block instances
  - Erases old FEE sector
  - At least two Flash sectors are necessary

1. Arrow indicates a link between two chunks.
2. In this example, each chunk consists of four chunk instances which are connected by a linked list.
3. A new list is created in a free Flash memory area after all four chunk instances have been filled with data.

cw 20 2023 ; VI-S CEA Seitech

Flash EEPROM Emulation (FEE) – key features

- Detection of incomplete writes
  (e.g., due to reset or due to cancellation)
- Combines multiple physical Flash sectors to logical FEE sectors
  → FEE always handles two logical sectors
- Logical sectors may differ in size
- Multiple partitions (in pairs of 2 FEE sectors) are supported for independent operation
- Redundant internal management information for increased robustness
- Configuration update supported
- Shared configuration usage between Flash Bootloader and application possible

CW_26_2023_VI-S CEA Seitech

1. At least two sectors are required. This allows to erase a completely filled sector while writing to the second one.
2. The latest data is copied from the previously used sector to the newly erased sector while switching between the Flash sectors.

Flash Driver (FLS)

- Provides access to Flash hardware
- Depending on the platform, Flash area(s) to be used can be configured

- Sometimes limited to address either Data Flash or Program Flash.
  - Some types of Program Flash cannot be written at the same time you execute the code from them
  - Normally the application software code resides in ROM (Program Flash)
  - Data Flash can typically be accessed at any time

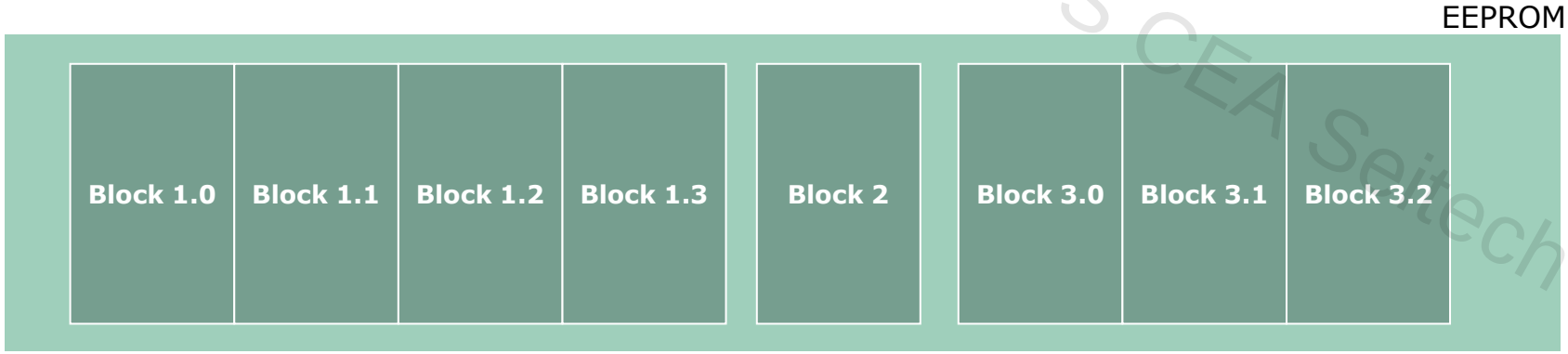
cw 26 2023 VI-S CEA Seitech

EEPROM Abstraction (EA)

EEPROM Memory

- Strategy to increase write cycles
  - Write one block to several memory locations in an alternating way

CW_26 2023 ; VI-S CEA Seitech



VECTOR

EEPROM Abstraction (EA) – Key features

- Detection of incomplete write operations by using header and footer
  - Caused by reset or cancellation
- ECC used for internally-added management information
  - Increased robustness and reliability
- "Walking concept" to increase number of write cycles
  - Each NV memory block consists of several instances
  - Written in a round-robin manner
- Automatic alignment of data supported in tool
  - Can speed up write operations
  - Avoids influences on other (adjacent) data blocks/instances

SW_26_2023 ; VI-S CEA Seitech

ECC: Error Correction Code – usually not provided by EEPROM hardware

ECC algorithm: combination of parity check and inverse management storage

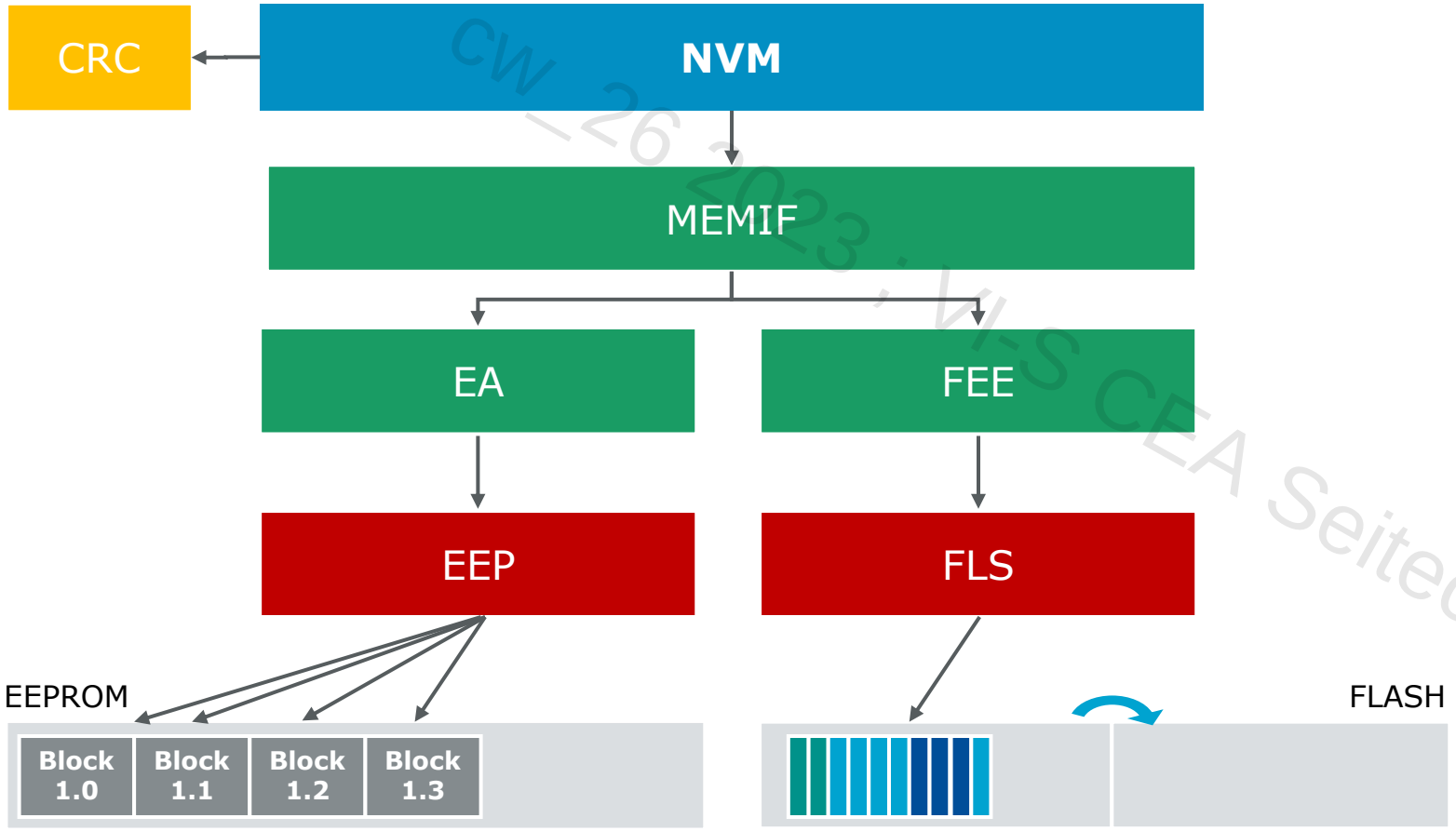
EEPROM Driver (EEP)

- Direct access to the EEPROM HW
- Provides byte-wise writing even if not supported by HW
- Write Cycle reduction (configurable)
  - Write only data if it is different from the current content
- Provides Erase service even if not supported by hardware
  - E.g., SPI EEPROMs don’t have an ERASE command
- Publishes device information that is necessary for the EA
  - Size of device, erased value (0x00/0xFF), writeable/erasable unit size

CW_26_2023_VKS CEA Seitech

Summary

BSW Modules Overview



CW_26_2023_WS CEA Seitech

CRC
CRC Routines
NVM
NVRAM Manager
MEMIF
Memory Abstraction Interface
EA
EEPROM Abstraction
EEP
Internal/External EEPROM Driver
FEE
Flash EEPROM Emulation
FLS
Internal/External Flash Driver

Agenda

- Principles of Memory Technologies
- The MICROSAR.MEM solution
- **Memory Handling**
- Configuration
- NvBlockSwComponents
- Service Mapping
- Coming up next

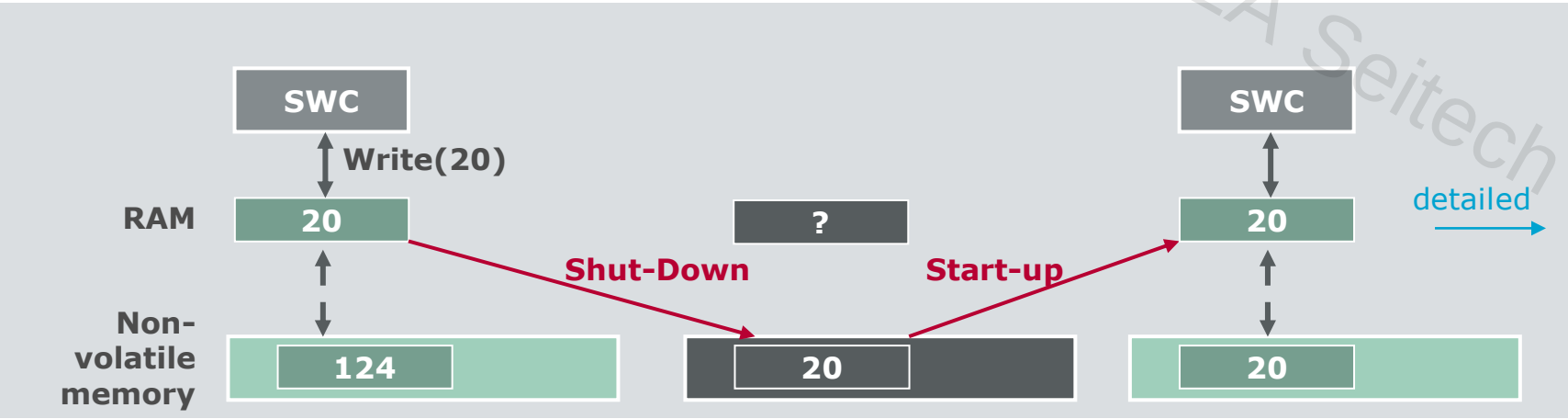
CW_26 2023 ; VI-S CEA Seitech

AUTOSAR Memory Abstraction

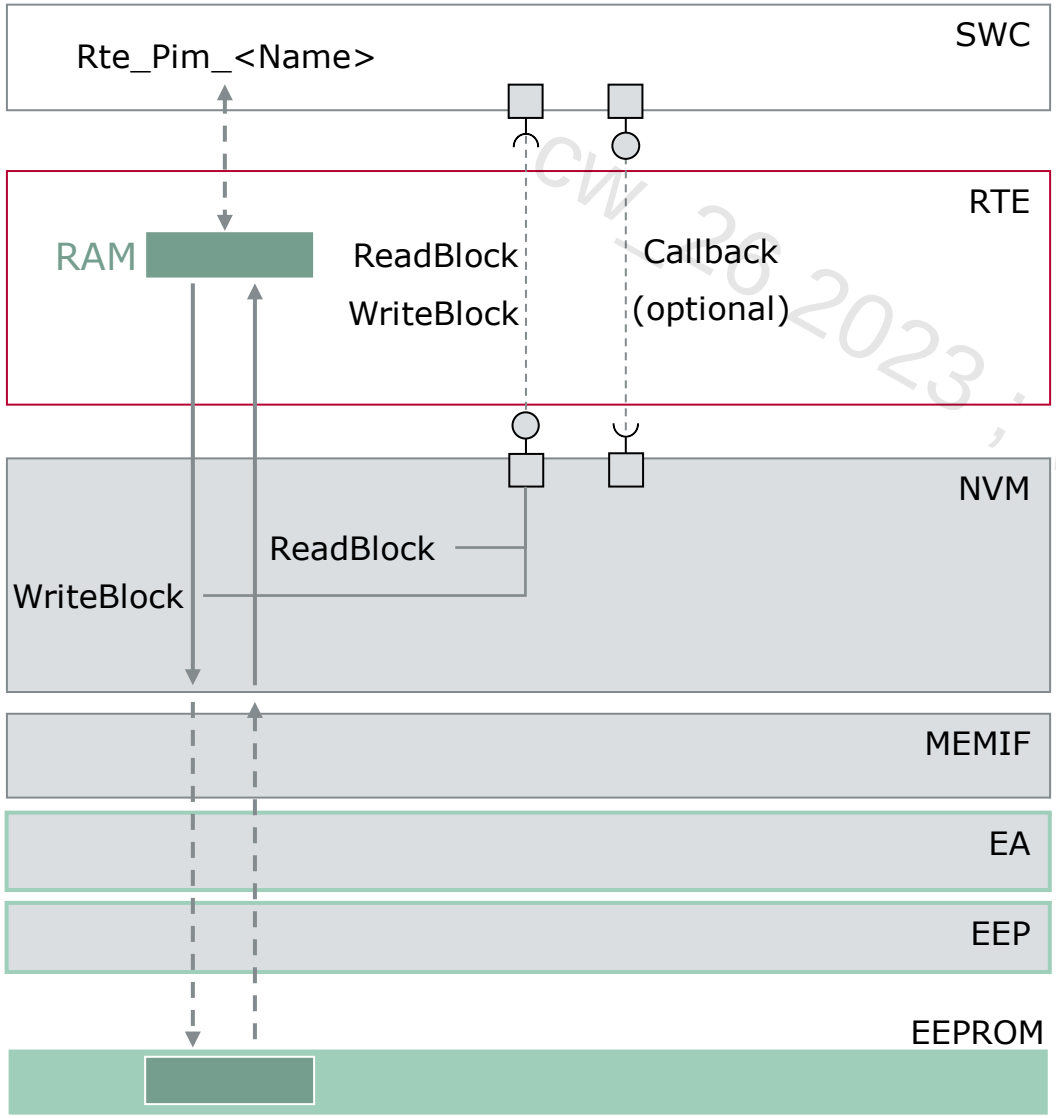
Reason for putting data into Non-Volatile (NV) Memory

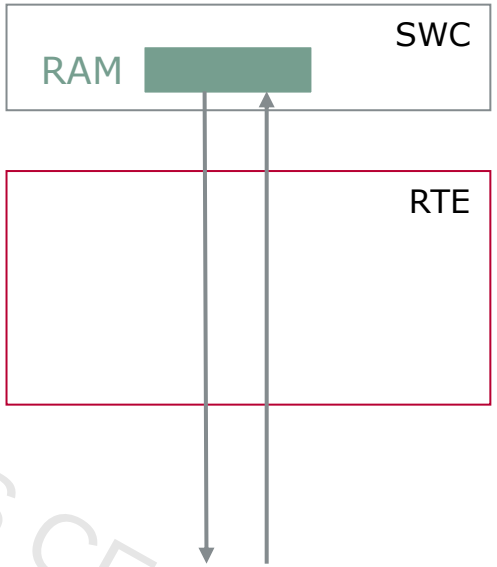
- Power-off data persistence
- Application works on RAM mirror of the NV data
- At startup data can be copied from NV memory to RAM
- At shutdown data is copied from RAM to NV memory
- Data can be read from or written to NV memory on request

CW 26 2023, VI-S CEA Seitech



Interaction with the NVM Module





Top: Application provides its own RAM mirror (SWC: global variable)

Left: RTE implements RAM mirror (RTE: per instance memory)

cw 26 2023 ; VI-S CEA Seitech

The NV Block

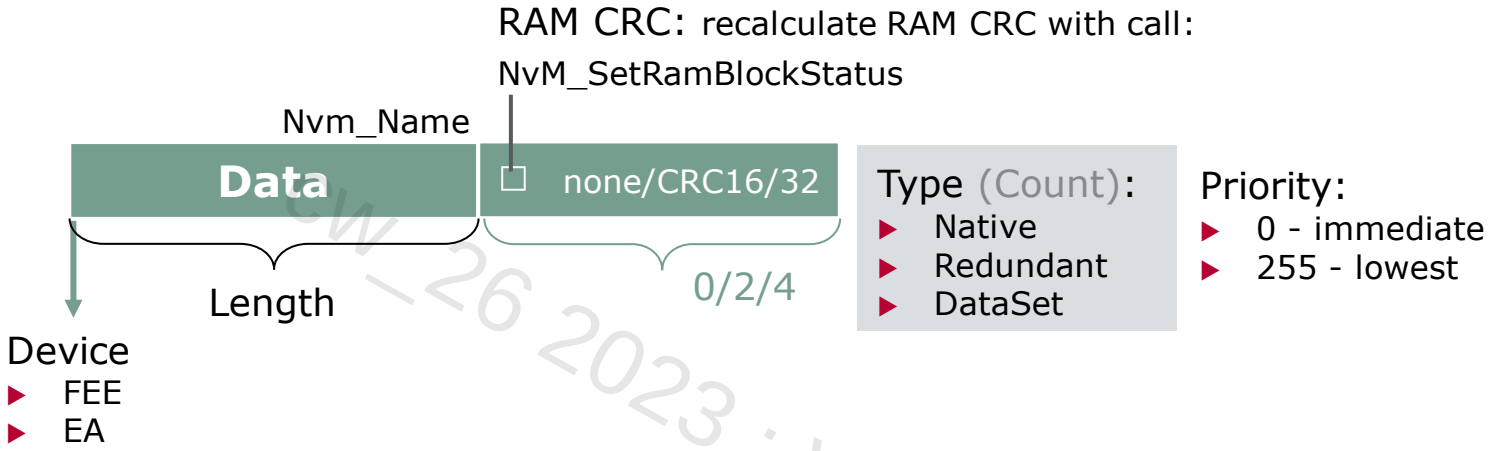
Base element is an NV Block

- It represents a memory area consisting of
  - NV user data
  - CRC value (optionally)
- Additional information necessary to define a Block
  - Name, Block ID, Size …

cw_26_2023 ; VI-S CEA Seitech

The way how the blocks are defined and administered in the NV memory can be configured. >>

NVM Block Parameters



Native

One NV block

1:1 mapping between RAM block and NV block

Redundant

Two NV blocks

Data is stored twice in NV memory

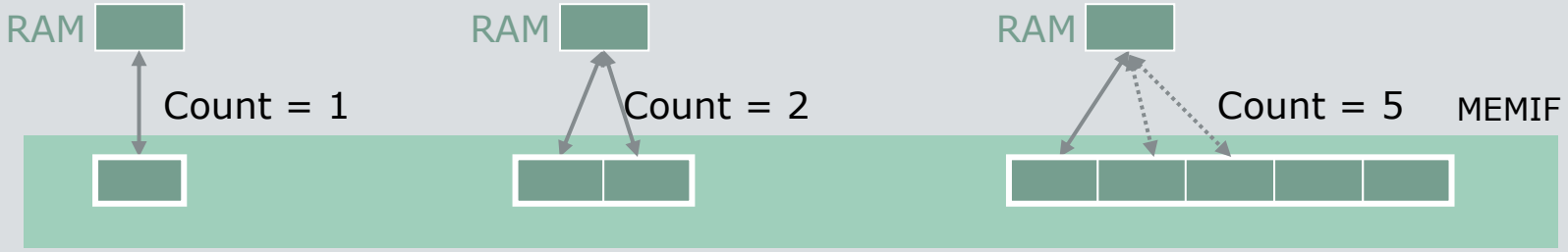
Second NV block will be read if the first one is defective (CRC error or not readable by underlying layer)

Dataset

Many (up to 255) NV blocks

1:n relation between RAM block and NV block

Read source/write target identified by a DataIndex (compare to an array in C)



cw_26 2023 ; vus_cfg_saitech

NVM Block Parameters

1. NvM_SetRamBlockStatus() triggers CRC calculation if configured and marks the RAM block as being changed and valid in order to trigger an NV block update during ECU shut-down
2. The currently used NV block for NVM **datasets** is configured by NvM_SetDataIndex()

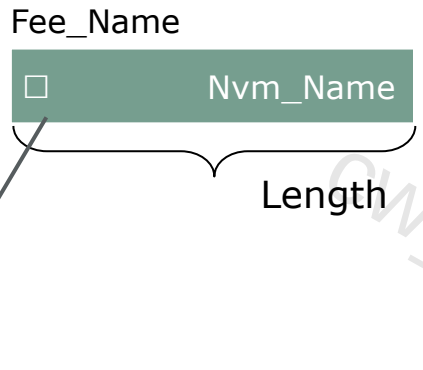
The (re-)calculation of a block's CRC is done asynchronously by the NvM_MainFunction(). A CRC protected block's CRC value is calculated every time the block shall be written to NV memory. If a block is read from NV memory the CRC value is recalculated and compared to the one that was just read from NV memory.

If NvM_SetRamBlockStatus(TRUE) is called, the calculation of the CRC value over the RAM block's data is also initiated, unless the configuration option 'Calculate RAM CRC' was disabled for this block. The purpose of requesting recalculation of the RAM CRC with every call to NvM_SetRamBlockStatus is to provide the possibility to reuse the RAM data even if a soft reset (short power loss, watchdog reset) occurs.

Since CRC calculation is quite time consuming, especially if performed frequently and/or over large data blocks, it might be more applicable to disable it using the option mentioned above. Then data can only be restored after a reset if it did not change since last CRC recalculation (caused by a write request or after the last successful read request). In case the CRC in uninitialized RAM still is correct, the NVM will not reload the corresponding blocks but instead use the RAM copy.

CW_26 2022, VIS OSEA Sentech

FEE Block Parameters



Chunk Block Count:
- 1, 3, 7, 15, 31, 63

Write Cycles:
- No effect on code
- Relevant for **FEE Optimization Assistant**

→ The chunk size should be increased if a higher number of write cycles is expected.

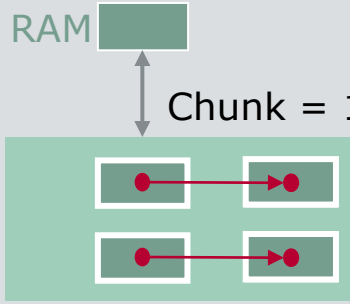
High Priority: this block must be written as fast as possible (e.g. crash data)

Organization of data blocks in "chunks" of configurable size (number of data block instances allocated at one time)

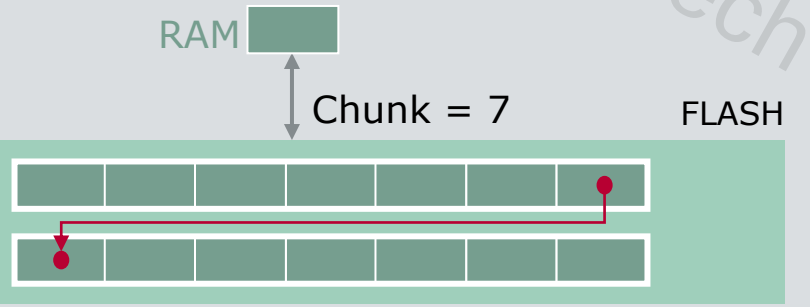
Chunks of one block form a linked list throughout the Flash memory sector

Binary search algorithm used within a chunk to increase performance

NVM Block: Redundant



NVM Block: Native



In MICROSAR FEE, the chunk size must always be a number equal to $2^n - 1$ because the FEE performs a binary search inside one chunk to retrieve the most recent data instance.

CW_26 2023 , VI-S CFA Seitech

Relation between NV Block and FEE Block

CFG NVM

Name	Crc usage	Ram Block Data	Datasets	Block Length
DemPrimaryDataBlock7	true	Dem_PrimaryEntry_7	1*	8
TestBlockOne*	false	RamSymbolOne	1*	16
TestBlockTwo*	false	RamSymbolTwo	1*	1*

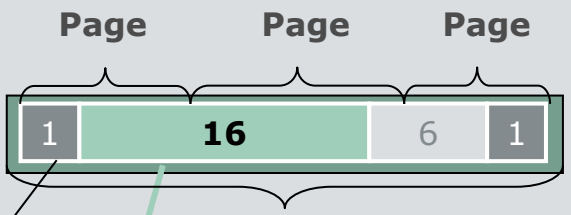
No CRC
Length = 16

CFG FEE

Short Name	Block Id Fixed	Block Size	Number Of Datasets	Number Of Instances In Chunk
FeeDemStatusDataBlock	false*	30	1*	1*
FeeTestBlockOne	false*	16	1*	3
FeeTestBlockTwo	false*	1*	1*	1*

Chunk = 3

Test Block Calculation

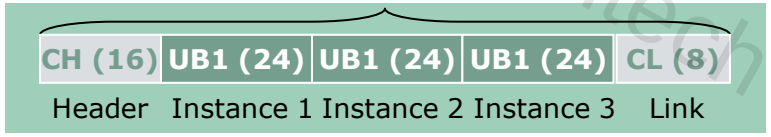


CFG: Fee_UserBlock1 (UB1)
24 byte

16 byte NVM payload

control information

FLASH **16 + 3x24 + 8 = 96 byte**



e.g. page size = 8

CW_26_2013 , VI-S CEA Seitech

Relation between NV Block and FEE Block

Since version 3.02.00 of MICROSAR NVM, the BSW module provides its own storage for the CRC value of RAM blocks. AUTOSAR originally required that the application has to include memory for the CRC with its own variable. This caused frequent problems as the additional RAM had not been provided by the application.

The chunk header (CH), chunk instance (CI), chunk link (CL) have to be aligned to whole pages individually. This does not cause any problems as long as the page size is smaller than and is a factor of the CH,CI, and CL sizes. If the page size is significantly greater than the CH, CI, and CL sizes, then there will be a lot of padding bytes, e.g., Infineon Tricore has a page size of 128 bytes. For the chunk link there would be 120 bytes of padding.

Rationale: The chunk header (CH) or chunk link (CL), as well as the chunk instance (CI) has to be written individually and atomically. This requires the usage of multiples of the pagesize for these individual writing operations for these parts.

CW_26_2023 : VLS CEA Seitech

Relation between NV Block and FEE Block

CFG NVM

Name	Crc usage	Ram Block Data	Datasets	Block Length
DemPrimaryDataBlock7	true	Dem_PrimaryEntry_7	1*	8
TestBlockOne*	true*	RamSymbolOne	1*	16
TestBlockTwo*	false	RamSymbolTwo	1*	1*

CRC32
Length = 16 + 4 = 20

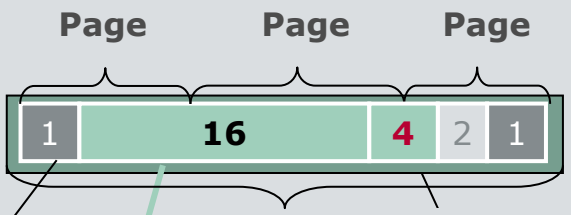
CFG FEE

Short Name	Block Id Fixed	Block Size	Number Of Datasets	Number Of Instances In Chunk
FeeDemStatusDataBlock	false*	30	1*	1*
FeeTestBlockOne	false*	20	1*	3
FeeTestBlockTwo	false*	1*	1*	1*

Chunk = 3

Since version 3.02.00 of MICROSAR NVM, the BSW module provides its own storage for the CRC value of RAM blocks. AUTOSAR originally required that the application has to include memory for the CRC with its own variable. This caused frequent problems as the additional RAM had not been provided by the application.

Test Block Calculation



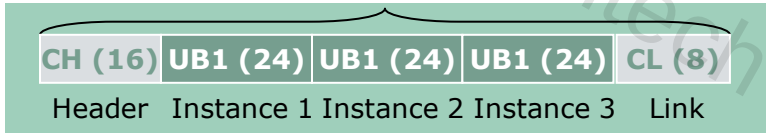
CFG: Fee_UserBlock1 (UB1)
24 byte

16 byte NVM payload

control information

NVM CRC32

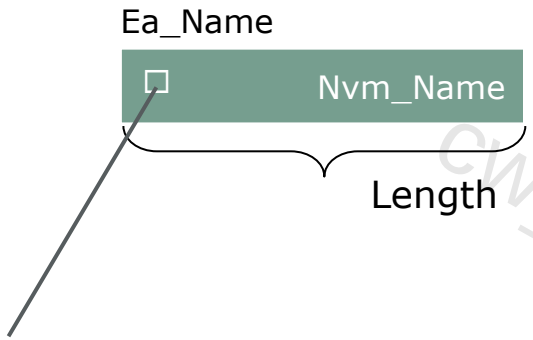
FLASH **16 + 3x24 + 8 = 96 byte**



e.g. page size = 8

CW_26_2013 , VI-S CEA Seitech

EA Block Parameters



Write Cycles:

- How often will this block be written during life time of the system

Verify Writing:

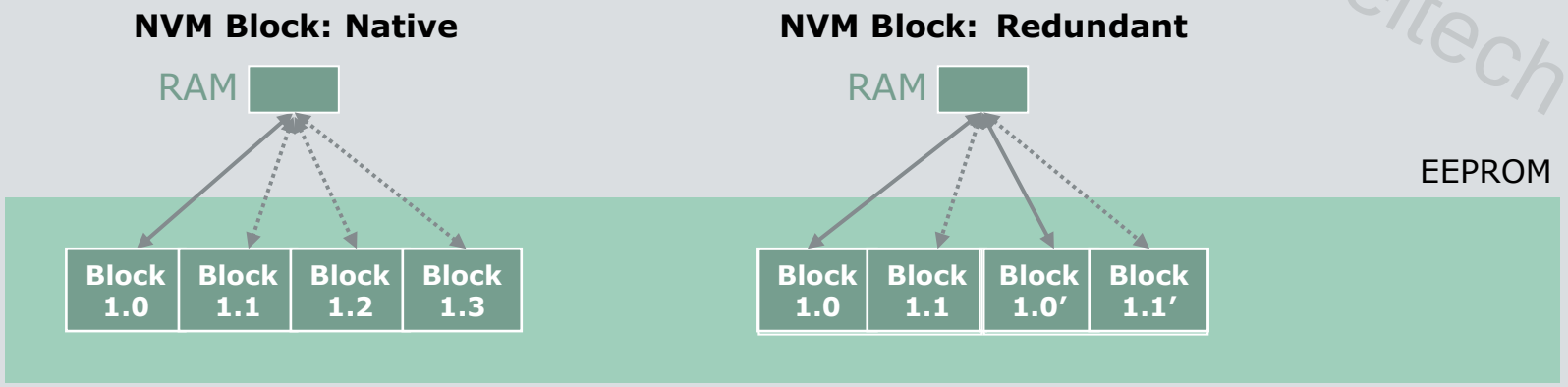
- Writing to EEPROM is verified or not
- Recommended for sensitive data

High Priority: this block must be written as fast as possible (e.g. crash data)

Information about how often memory can be rewritten

Physical write cycle value is basis for calculation of necessary instances of the EA block in the EEP

At runtime these blocks are written alternately



cw_26 2023, VI-S CEA Seitech

Relation between NV Block and EA Block

CFG NVM

Insert Block | Delete Block | Move Up | Move Down

#	Name	Device	Mngmt Type	CRC Type	RAM CRC	Prio	Length	Count
2	TestBlockTwo	Ea	NATIVE	CRC16	✓	127	10	1

Length = 10

CRC16
Length = 10 + 2 = 12

EA

Insert Block | Delete Block | Move Up | Move Down | Calculate

NvM Blockn...	Ea BlockName	Block Num	Block Addr	Datasets	Size	Write Cycles	High Prio	Verify Writing
TestBlockTwo	Ea_UserBlock2	0x02	0x00	1	12	10000	☐	☐
	Ea_TestBlock	0x04	0x10	1	4	10000	☐	☐

Insert Block | Delete Block | Move Up | Move Down | Calculate

NvM Blockn...	Ea BlockName	Block Num	Block Addr	Datasets	Size	Write Cycles	High Prio	Verify Writing
TestBlockTwo	Ea_UserBlock2	0x02	0x00	1	12	50000	☐	☐
	Ea_TestBlock	0x04	0x20	1	4	10000	☐	☐

cw 08 2023 ; VIS CEA Seiteen

Instances ≈ Write Cycles / max. possible cycles

Write Cycles = 50000 >= ½(max. possible cycles)

One Instance

1	10	2	1	2

0x10 (16 byte)

control information

padding
virtual page size

1	10	2	1	2
1	10	2	1	2

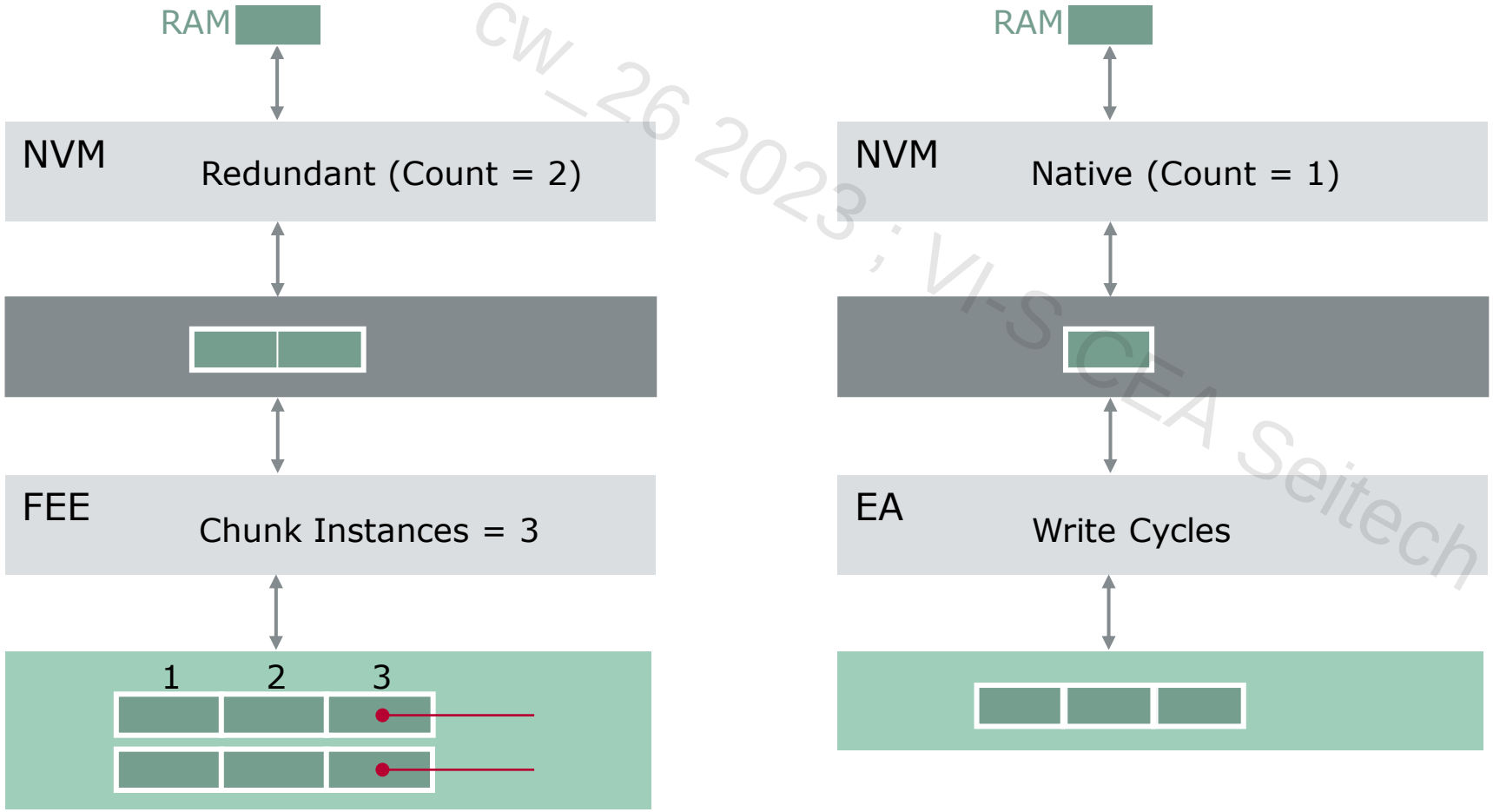
Control information

0x20 (32 Bytes)

Alternative Block to realize requested write cycles

Remark: Screenshots not taken from most recent tool (legacy toolchain GENy + Configurator 4)

Summary



cw_26 2023 ; VI-S CEA Seitech

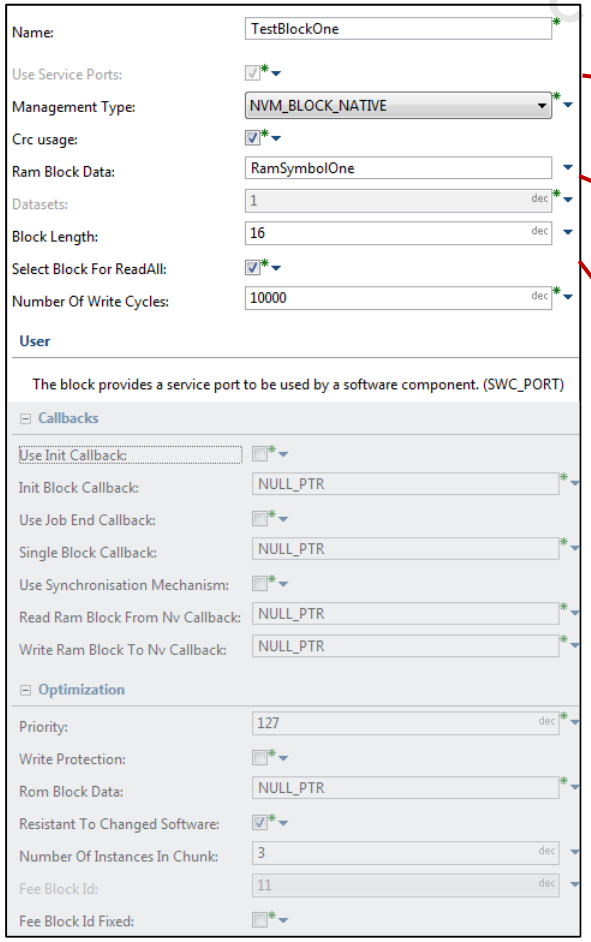
Agenda

- Principles of Memory Technologies
- The MICROSAR.MEM solution
- Memory Handling
- **Configuration**
- NvBlockSwComponents
- Service Mapping
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

NVM: Fundamental Settings

Memory Blocks



Service Ports

- Generate a service port description

RAM Block

- Enter Name for RAM Block in Application
- Enter NULL_PTR if no permanent RAM block is required
- E.g., name of the Rte Per-Instance Memory

ReadAll

- Block loaded by NvM_ReadAll at startup

CW 26 2023 ; VI-S CEP Seitech

NVM: Fundamental Settings

Diagnostic data does not need to be marked as ReadAll. It can be read on request to reduce ECU startup time.

The feature “Resistant against changed configuration (Res)” works based on the configuration ID value.

If this ID is increased each time the configuration (application) is changed, the NVM can detect configuration changes.

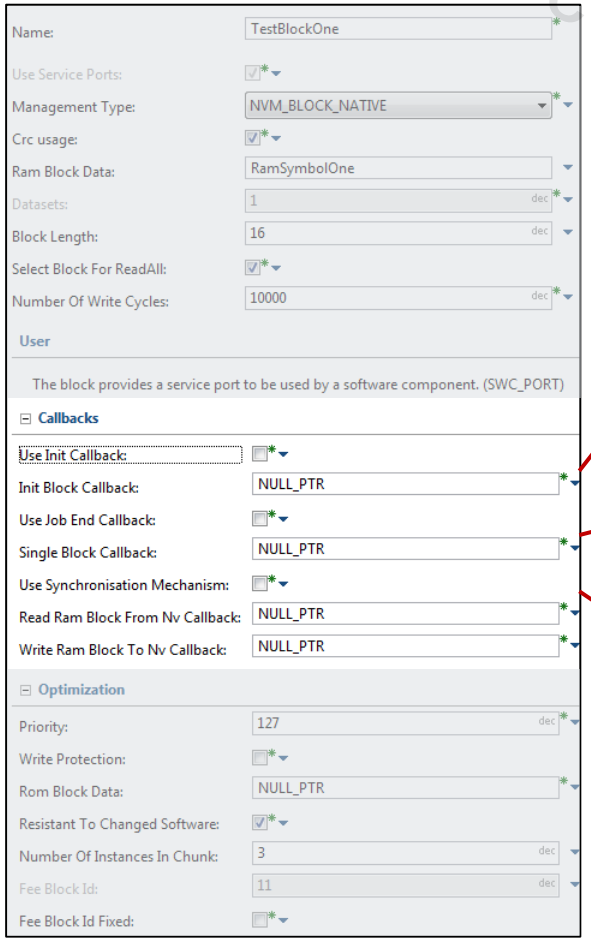
Then, blocks containing the “Res” bit can be retained in the application, provided that the NV Block structure has not changed.

This is intended to conserve NV data like odometer counters, diagnostic data, or seat memory settings throughout the life cycle in the field, without changing due to software updates performed by service.

cw 26 2023 ; VKS CEA Seitech

NVM: Callback configuration

Further Settings



Init Block Callback

- Use configured init callback to provide default data
- Callback is called if no "ROM Block" is configured to initialize this block (NULL_PTR)
- NULL_PTR: no callback function to be called

Use Job End Callback

- Use the configured callback for single block job end notification, either directly or via Service Port (RTE) depending on the 'Use Service Port' setting

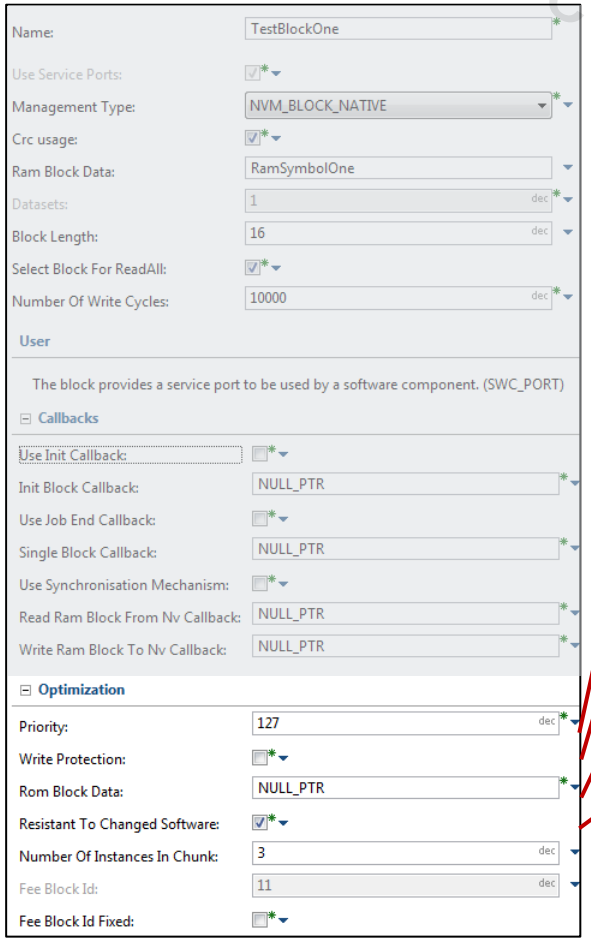
Explicit Synchronization

- RAM mirror in the NVM module. The data is transferred to/from the application in both directions via callback routines called by the NVM module.

cw_26_2023 i-VI-S CEA Seitech

NVM: Optimizations

Further Settings



Priority

- Priority in the Job Queue

Write Protection

- Software Write Protection
- Write protection can be removed by API

ROM Block Data

- Start Address (symbol name) of ROM Block data
- Name of the RTE calibration data

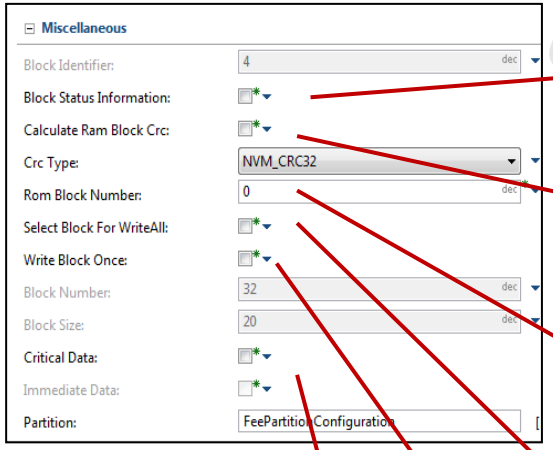
Res (Only for ReadAll)

- Resistant against changed configuration
- Preserve old user settings

CW_26 2023 ; VRS CEA Seitech

NVM: Miscellaneous

Further Settings



Block Status Info

- Defines whether BswM is informed about the current status of this block

Ram Block CRC

- Calling `Nvm_SetRamBlockStatus` retriggers RAM Block calculation

Rom Block Number

- Defines the number of multiple ROM blocks in a contiguous area according to the given block management type.

Write All

- Write on `NvM_WriteAll` call

WriteOnce

- Only write this block once

Critical Data / Immediate Data

- Handle data loss in exceptional situations (especially Sector Overflow)
  TRUE: always keep most recent data (frequently written data is not critical)
- Immediate Data marks high priority data for a different job queue

CW_26_2022, VI-S CEA Seitech

Agenda

- Principles of Memory Technologies
- The MICROSAR.MEM solution
- Memory Handling
- Configuration
- ▶ **NvBlockSwComponents**
- Service Mapping
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

Introduction to NvBlockSwComponents

- AUTOSAR 3.x: NVRAM handling is block oriented
  - Management at the NV block level is not sufficient especially if small data has to be stored individually
    - Inefficient usage of paged flash memories
  - Distribution of non-volatile data between SWCs is difficult to handle
    - PIMs or module-local static variables are private members of an SWC
  - Access over PIMs is not protected against concurrent access
- AUTOSAR 4.x addresses this situation by introducing optional **NvBlockSwComponents**

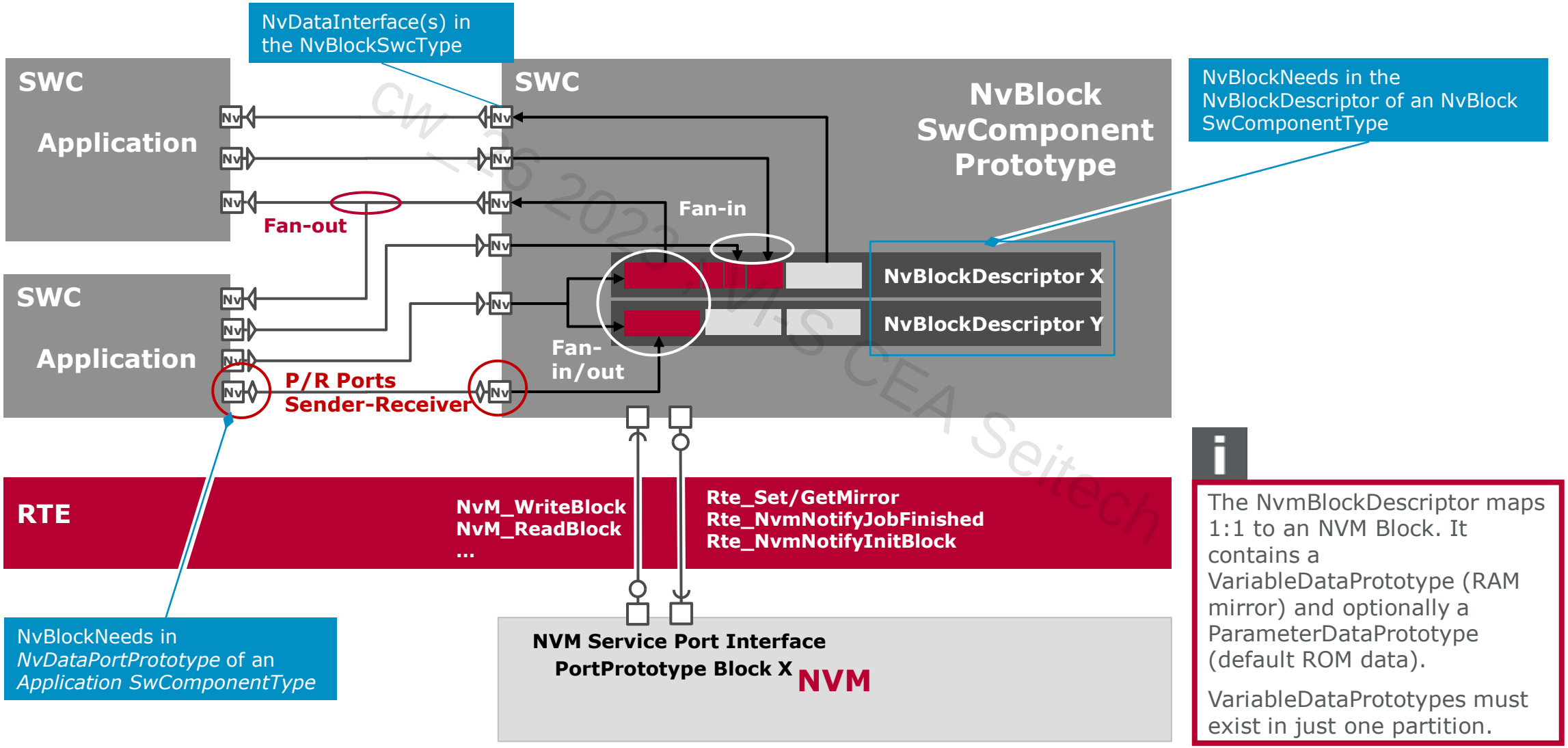
cw 39 2023 . VI-S CEA Seitech

Introduction to NvBlockSwComponents

- S/R oriented NVRAM handling
- Aggregation of several RAM variables into one NVRAM block (analogous to Signals and Signal Groups in Com)
- “Fan-in” and “fan-out” of data elements to and from NVRAM blocks
  - Additional RAM is required because an extra buffer stage is required for handling the “fan-in”
  - “Fan-out”: overall RAM consumption may be lower because the same data elements can be shared in the ECU with this feature (like calibration data)
  - NvData elements can fan into/out of the same (shared) NVRAM Block Element
- Flexible NvData distribution across multiple SWCs
- Protection against concurrent access by RTE

CW_26_2023, VKS CEA Seitech

Introduction to NvBlockSwComponents

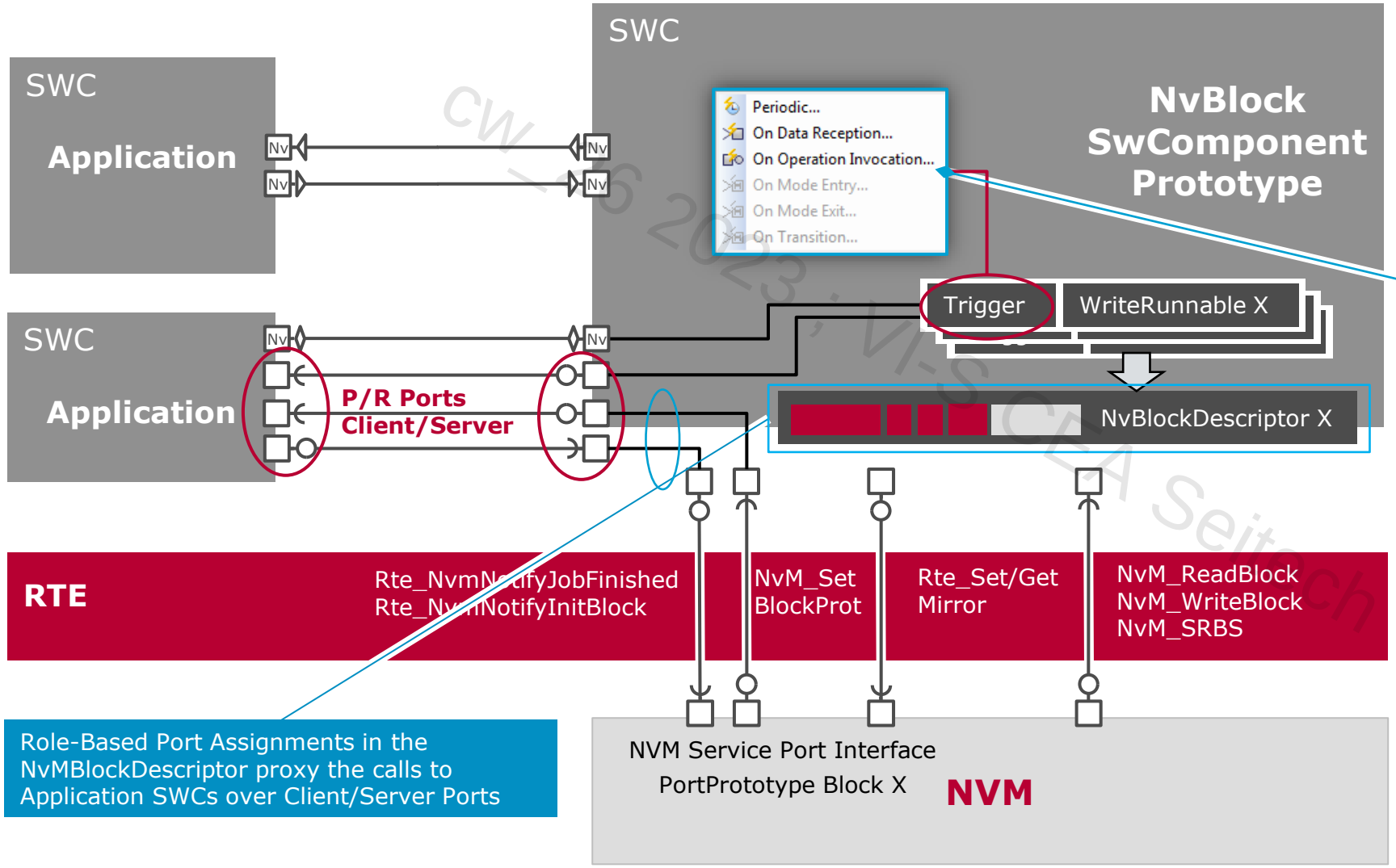


The NvmBlockDescriptor maps 1:1 to an NVM Block. It contains a VariableDataPrototype (RAM mirror) and optionally a ParameterDataPrototype (default ROM data).

VariableDataPrototypes must exist in just one partition.

CW_AS_2022_WS CEA Seitech

Control Flow



Triggers for the Write Runnables originate from NvDataPorts or Server Ports

Role-Based Port Assignments in the NvMBlockDescriptor proxy the calls to Application SWCs over Client/Server Ports

The Service Port Prototypes indicated on the illustration need not be created manually. They are created automatically and are not visible in the ordinary way.

As opposed, the Application Port Prototypes used to control the NvBlockSWC have to be created manually. The Port Interfaces for them must be taken from the SWCServiceRelatedInterfaces blueprints.

cw_KW 2023, VI-SCEA Seitzch

NV Data Ports and their relation to NV Block Descriptors

- In AUTOSAR, the NVData elements can already carry contributions to the Service Needs
- If there are several NVData elements which are combined in to one NvBlockDescriptor, there have to be arbitration rules on how the NVBlock in the NVM must be configured.
  - Example: NVM Parameters
    1. Writing frequency
    2. RAM Block CRC

The highest writing frequency among all NV data ports determines the configuration value for the NvBlock in the NVM

NvBlockNeeds attrib.	@NV data Ports	@NvBlockDescr.
Readonly	=	=
Reliability	!=	=MAX
Resistant to change	=	=
Restore at startup	=	=
Store at shutdown	=	=
writeOnlyOnce	=	=
writingFrequency	!=	=MAX
writingPriority	!=	=MAX
writeVerification	!=	OR
calcRamBlockCrc	!=	OR
checkStaticBlockId	!=	OR
ramBlockStatusControl	!=	OR

If any NV data Port requires a CRC protection, the whole NvBlockDescriptor has a service need / NvBlock in the NVM has to support CRC.

Legend:

'=': recommended to be identical

'=MAX': recommended to be the maximum of all existing values

'OR': recommended to be set TRUE if at least one of existing values of NV data Ports is set TRUE.

'!=': may be set differently among different NV data Ports

©VIS 2023 ; VI-S CR4 Seitech

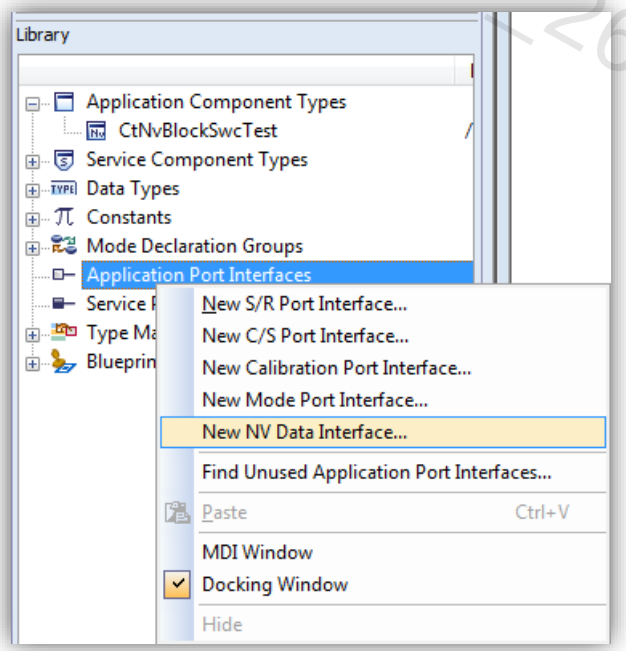
Implementation of NvBlockSwComponents

- SWC type with interface to NVM
  - Support for NvMAdmin, NvMService, NvMJobFinished, NvMNotifyInitBlock
  - The NVM API can be operated through the NvMBlockSWC directly
  - Client/Server and ModeSwitch interfaces
  - Multiple NvBlock to port mappings for dual-sided RAM block access
  - Explicit modelling of different writing strategies
    - Periodic, On Data Reception, On Operation Invocation
    - On Mode Entry, On Mode Exit, On Transition

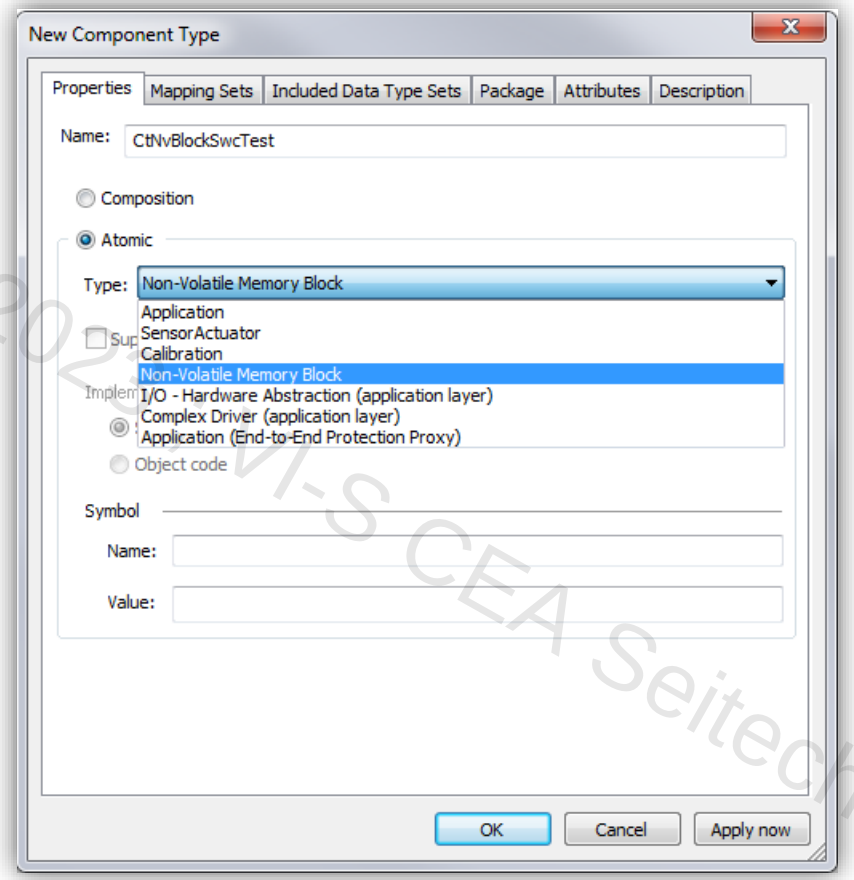
CN_26_2023, VI-S CEA Seitech

Implementation of NvBlockSwComponents

- SWC type Equipped with P/R Nv Data Interfaces



DaVinci Developer: NV Data can be described using NV Data Interfaces. This is similar to the S/R communication paradigm.



DaVinci Developer: A new Nv Block SWC can be created by choosing Non-Volatile Memory Block as the Atomic SWC Type. This SWC Type is created automatically and only treats the NV Data and has no separate functionality.

NV Block Data Mapping

The NvBlockDataMapping specifies the mapping of DataElements of a P/R PortPrototype to DataElements inside the RAM block.

Multiple mappings to the same RAM block element are possible.

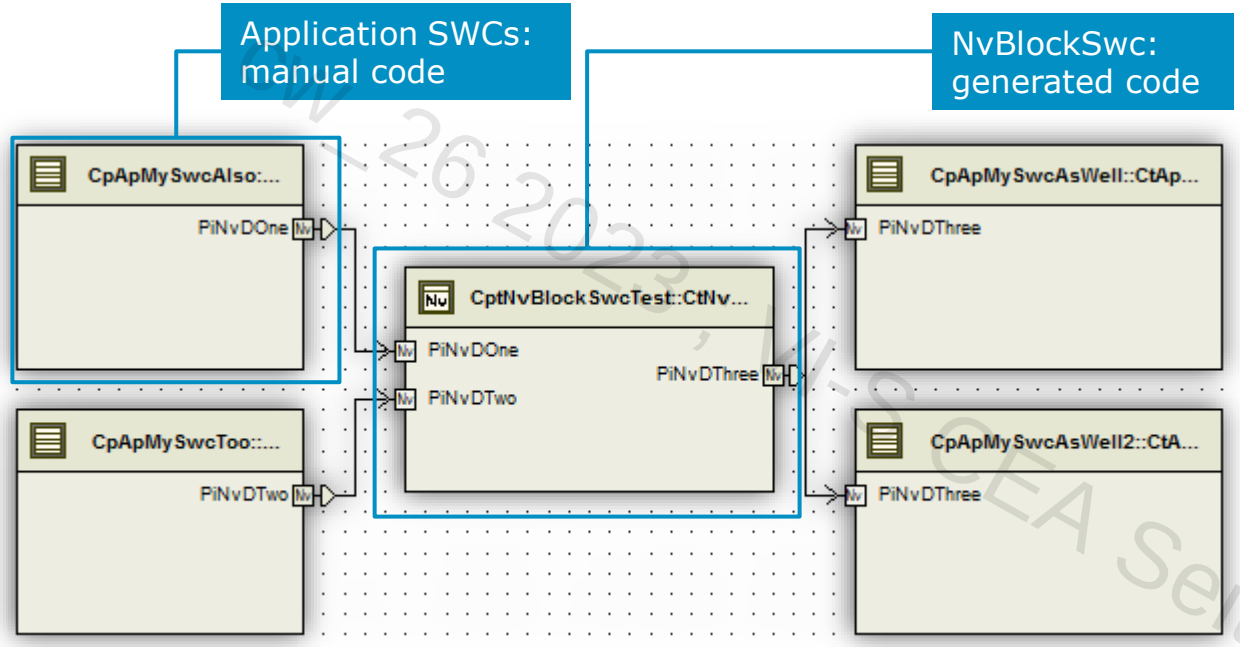
When data is written to a RAM Block, the corresponding data comes from an R-Port that is connected to an application component.

If data is read from the RAM block this data is sent through a P-Port to the connected components.

cw_26 2025 VI-S CEA Seitech

Implementation of NvBlockSwComponents

Example Software Design



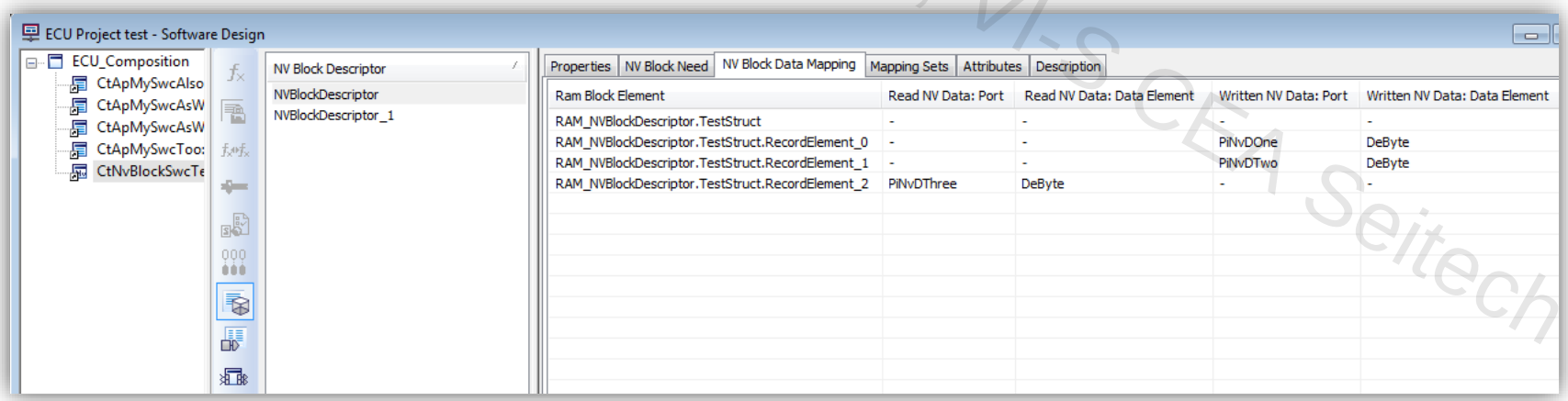
DaVinci Developer: The Software Design shows NV Data Port Prototypes using the "Nv" label in the port pictogram. Arbitrary SWCs can use the NvBlockSwComponent, which manages the NVRAM Blocks.

CW 26 2023, Vis CEA Seitech

NV Block Data Mapping

- Internally, the SWC must have at least one NvBlock Descriptor
- The NvBlock Descriptor is used to describe the RAM mirror
  - Fan-in is only possible if the NvBlock descriptors is a complex data type
  - Fan-out is always possible since the distribution of a data element will be done by the RTE Provider (Sender) Port

CW_26 2023 ; VI-S CEA Seitech

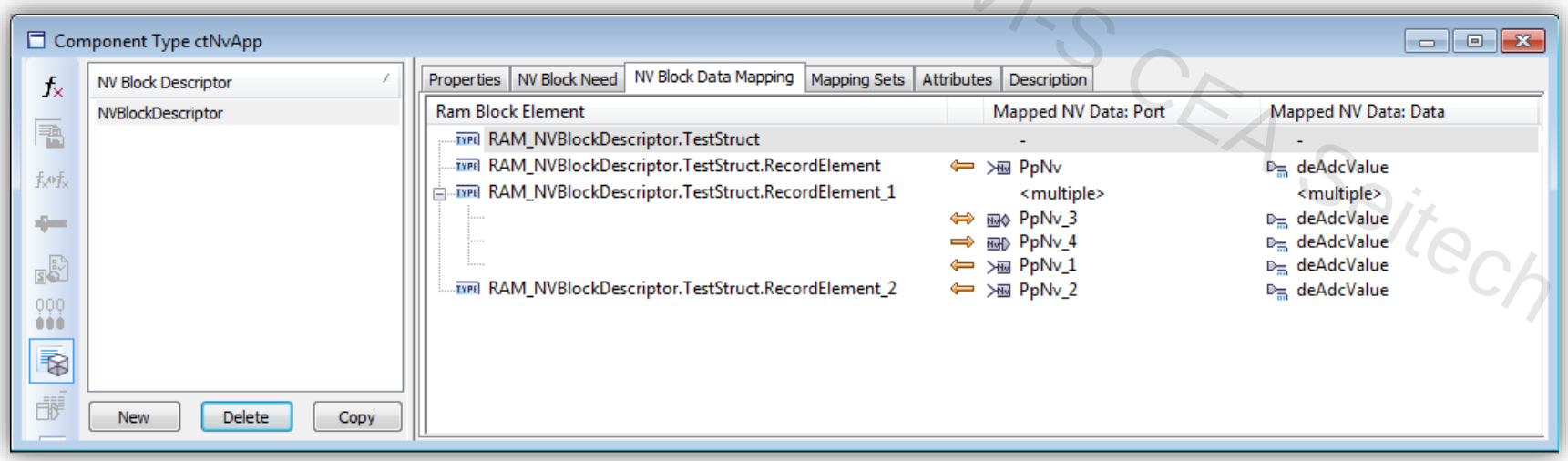


DaVinci Developer: NV Block Data Mapping: NVBlockDescriptors are the link between NvData Ports and the NVRAM Block in NVM. If a fan-in is required, the NVBlockDescriptor must have a complex data type (MICROSAR4 Release 9 shown).

NV Block Data Mapping

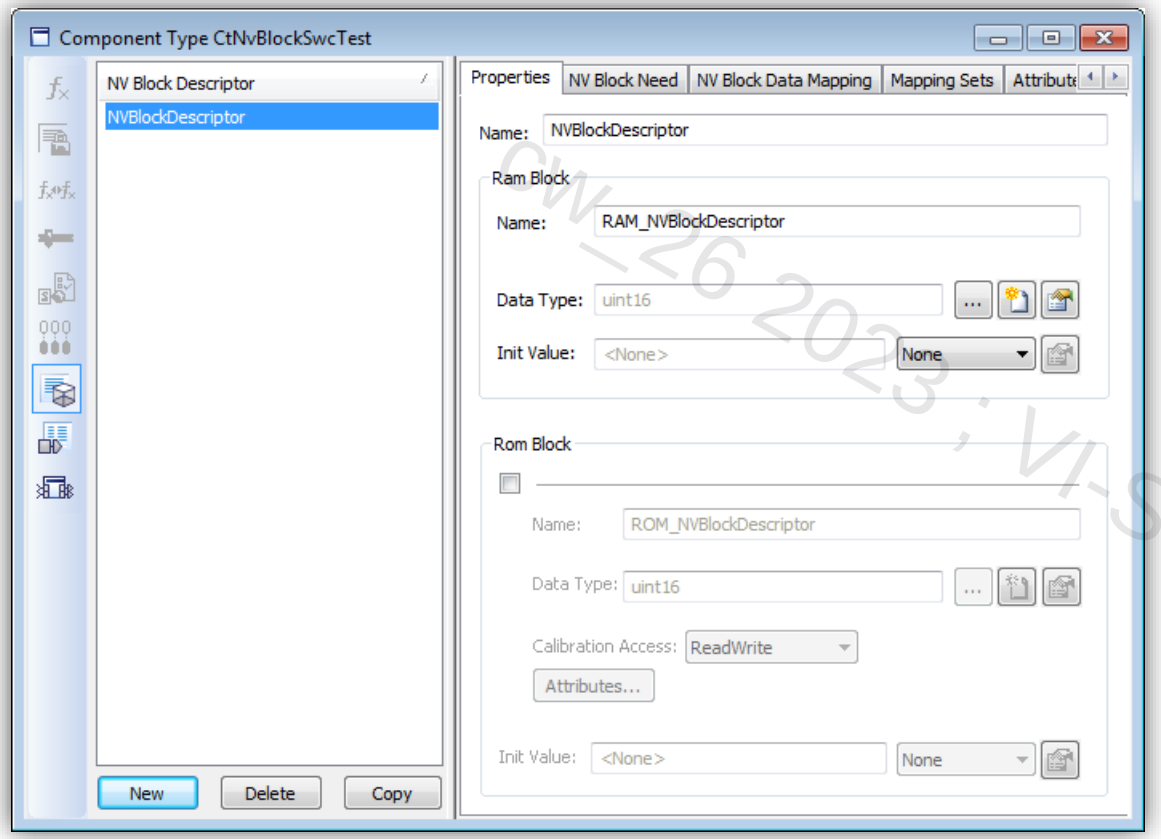
- Specifies the mapping of DataElements of a P/R PortPrototype to DataElements inside the RAM block
- Multiple mappings to the same RAM block element are possible
- When data is written to a RAM Block, the corresponding data comes from an R-Port that is connected to an application component
- If data is read from the RAM block this data is sent through a P-Port to the connected components

CW_26_2023 ; VI-S CEA Seitech



DaVinci Developer: NV Block Data Mapping: One RAM block element can be assigned to several Nv Data Ports / Elements. You can recognize that from the "<multiple>" string in the table. Please also note that S/R and SR Ports are supported (MICROSAR4 Release 13 shown).

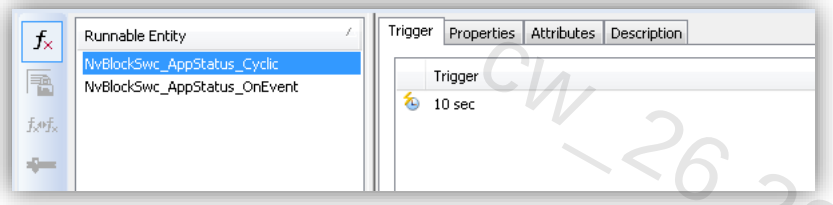
Properties of the NV Block Descriptor



CW_26_2023 ; VI-S CEA Seitech

DaVinci Developer: The NV Block Descriptor defines the data type(s) which may be used for a NVBlock Data Mapping. Here only an atomic data type 'uint16' is used. But we may also use a record (C struct) here to allow for efficient NVRAM Block configurations. The NV Block Descriptor has a 1:1 mapping to NVRAM Block.

Write Runnable definitions



Definition of **Runnables**

- Periodic trigger
- On Data Reception trigger

CW_26 2023 ; VI-S CEA Seitech

Properties of the NV Block Descriptor

Component Type ctNvApp

NV Block Descriptor
NVBlock_AdcValue

Properties | NV Block Need | NV Block Data Mapping | Mapping Sets | Attributes | Description

Name: NVBlock_AdcValue

Ram Block
Name: RAM_NVBlock_AdcValue
Data Type: uint16
Init Value: 0 Numeric
Attributes...

Rom Block
Name: ROM_NVBlock_AdcValue
Data Type: uint16
Calibration Access: ReadWrite
Attributes...
Init Value: <None> None

Support Dirty Flag

Check Dirty Flag

Writing Runnables
Periodic Trigger
<None>

Select periodic trigger from existing Runnables

Data Reception Triggers

Trigger	Runnable
piAdcValue.deAdcValue	NvM_AdcValue_IoHwAb_Get_Adc_Si...

Display all existing triggers and runnables for NvData elements within this Block Descriptor

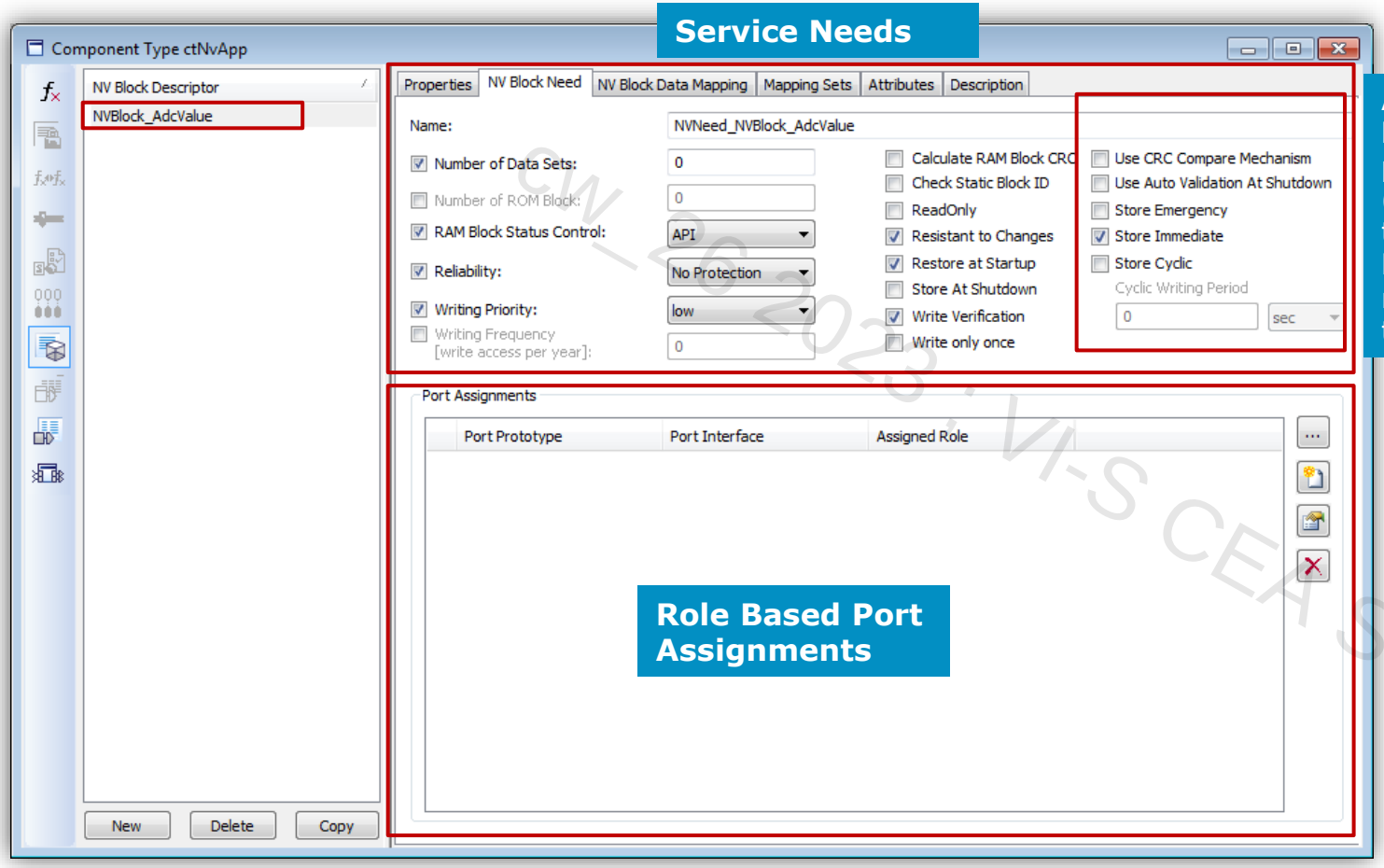
New | Delete | Copy

CW_26_2023_VW_CEA_Seitech

DaVinci Developer: The linkage between the Write Runnables and the NvBlockDescriptors is established by the data receive triggers of NvData Elements. These Elements are mapped onto NvMBlock Descriptors. The Triggers for the Write Runnables are displayed. Additionally, it is possible to define a cyclic writing activity using a periodic trigger (of a Writing Runnable)

Support Dirty Flag
Specifies whether the RTE shall control the calling of NvM functions for writing and/or status control for potentially modified RAM Blocks

Service Needs (NvBlockNeeds)

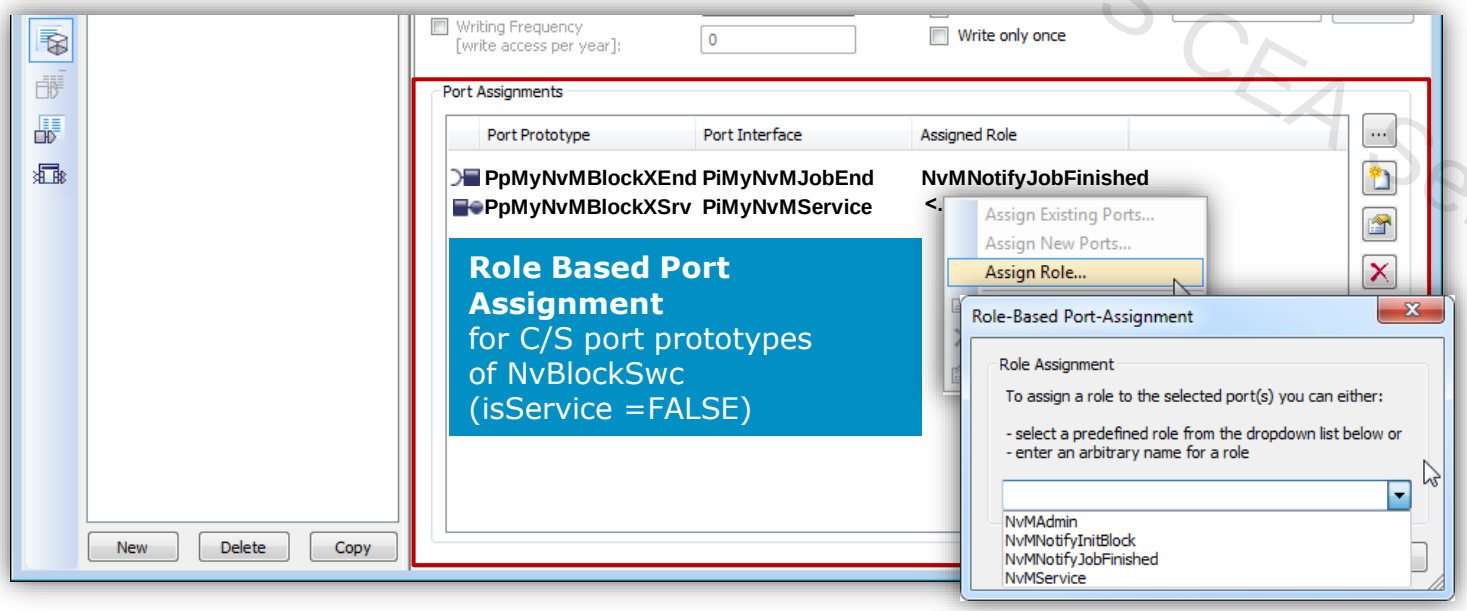


Additional **block need parameters** (requirements to be fulfilled by the runnable triggers)

CW_26_2025 · VI-S CEA Seitech

Role-Based Port Assignments (part of Service Needs)

- Per NvBlockDescriptor, the Port Assignments define which Client/Server port(s) of the NvBlockSwComponentType are associated with what kind of service or notification
- For notifications, the RTE provides one common callback function for each individual kind of notification defined by the "role"
- Role-Based PortAssignments
  - Client Ports: NvMNotifyJobFinished, NvMNotifyInitBlock
  - Server Ports: NvMAdmin, NvMService



cw_26_2023 ; VI-S CEA Seitech

Role-Based Port Assignments (part of Service Needs)

- Support of C/S Interfaces for Nv Block SWCs
  - In addition to NV-Ports, there are
    - Server Ports
    - Client Ports
    - Mode R-Ports
  - Only **<u>application</u> port interfaces** can be selected
- What is the exact mapping between a C/S Port and a Port Assignment?
  - For example, how does the RTE know which function of the **NvMService Port** maps to the **custom C/S Port**?
  - Use compatible port descriptions (blueprints)
    - Available at www.autosar.org
    - AUTOSAR_MOD_SWCServiceRelatedInterfaces_Blueprint.arxml (part of AUTOSAR_MOD_GeneralBlueprints.zip)
- Also supported in CFG5
  - Display the internal behavior of NvBlock SWCs

cw_26_2023 - VT-S CEA Seitech

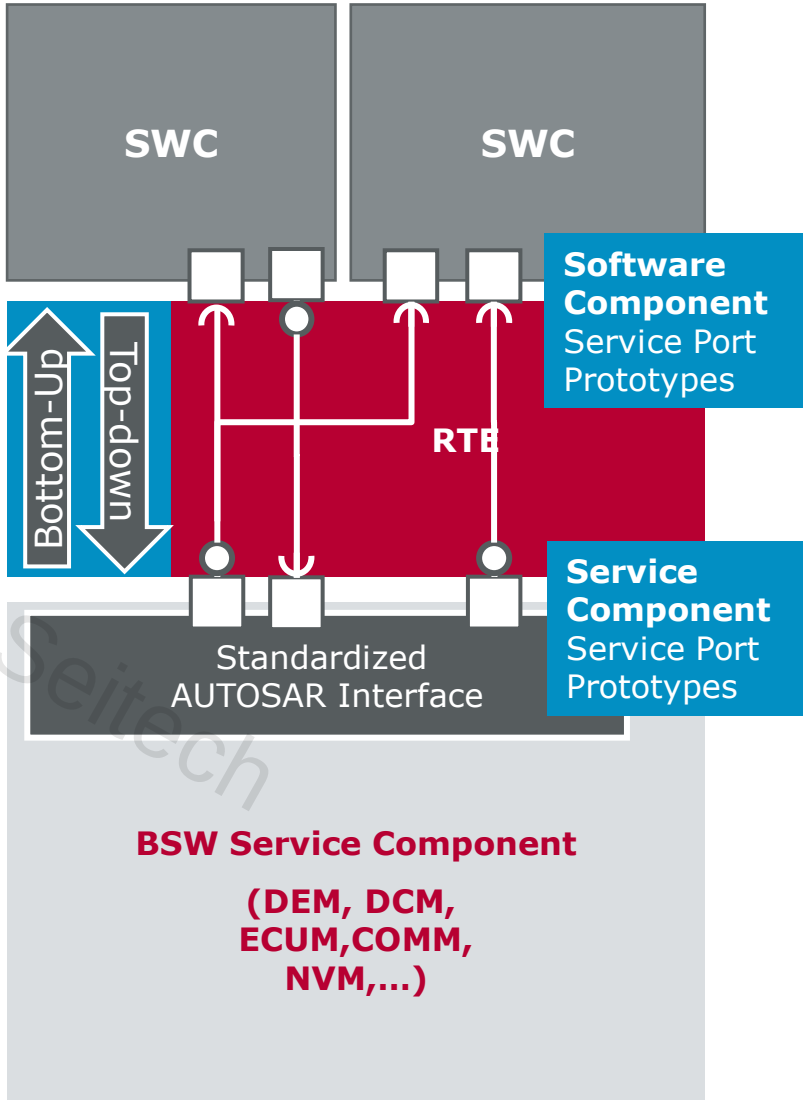
Agenda

- Principles of Memory Technologies
- The MICROSAR.MEM solution
- Memory Handling
- Configuration
- NvBlockSwComponents
- ▶ **Service Mapping**
- Coming up next

CW_26 2023 ; VI-S CEA Seitech

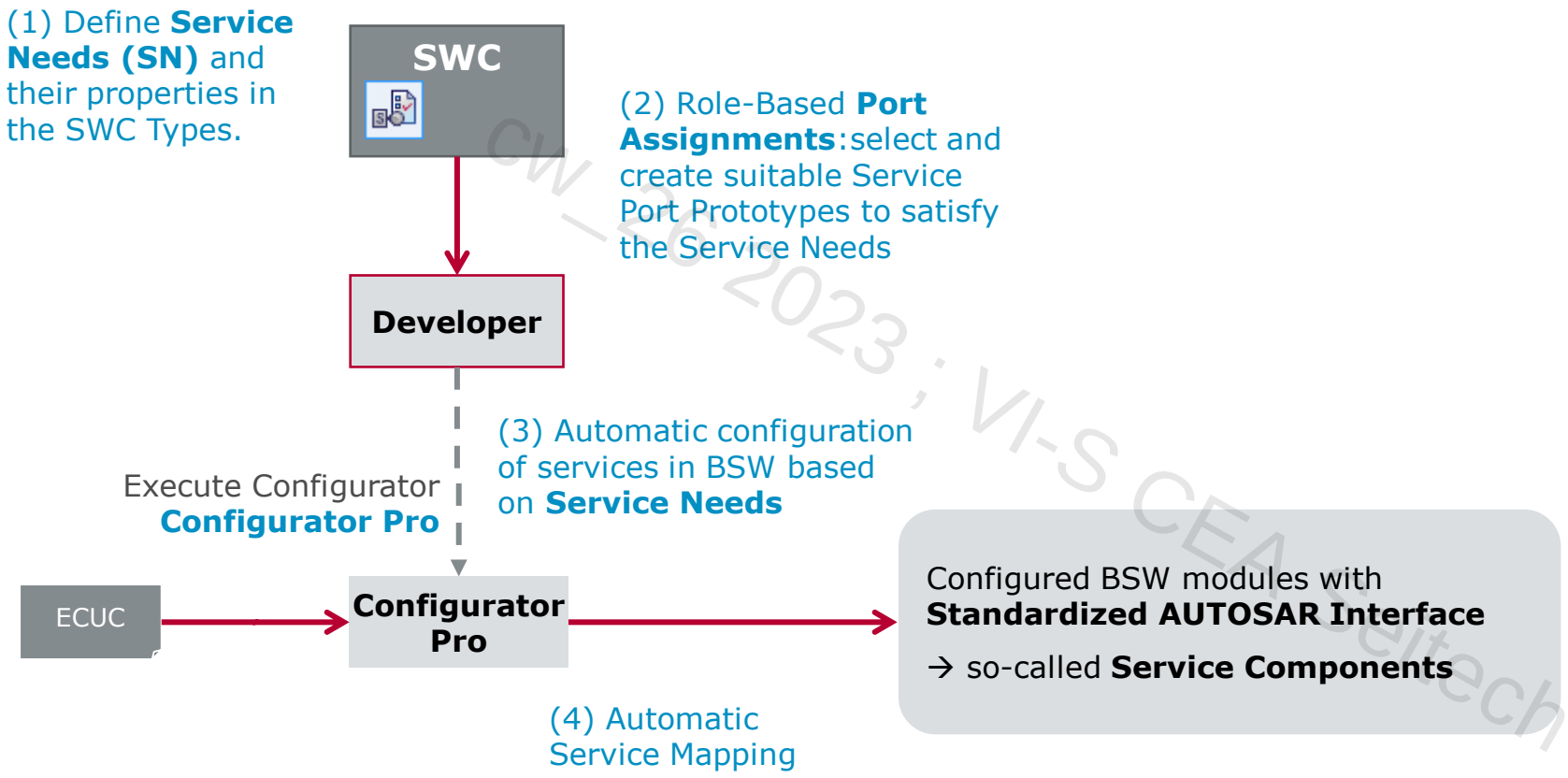
Service Mapping

- Assigning Service Ports of Software Components to Service Ports of Service Components or vice versa
- Bottom-up service configuration
  1. Configure a service module in the basic software
  2. Assign the respective service port prototypes to the SWCs where appropriate
- Top-down service configuration
  1. Use Service Needs
     - Document at the SWC level what is required from the BSW as a service
  2. Derive a service module configuration based on these requirements
  3. Configure the basic software service module in detail



cw 26 2023, VI-S CEA Seitech

Service Component Configuration (one-stop solution)



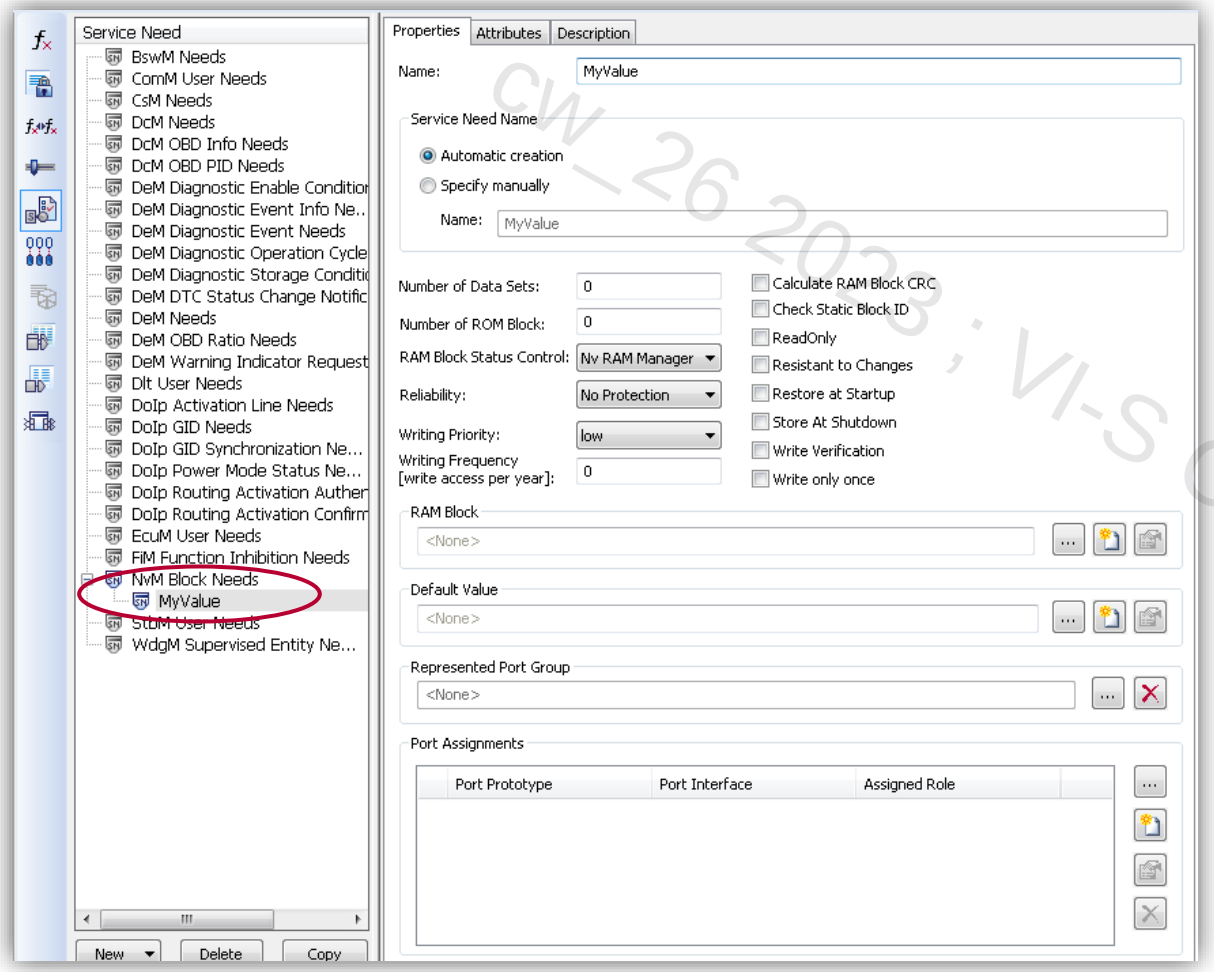
cw_26_2023 ; VI-S CEA Seitech

Top-down

In DEV you define **Service Needs** and create **Service Port Prototypes** at the SWC. CFG creates **configuration** and **Service mapping**

Top-down service configuration (NvM)

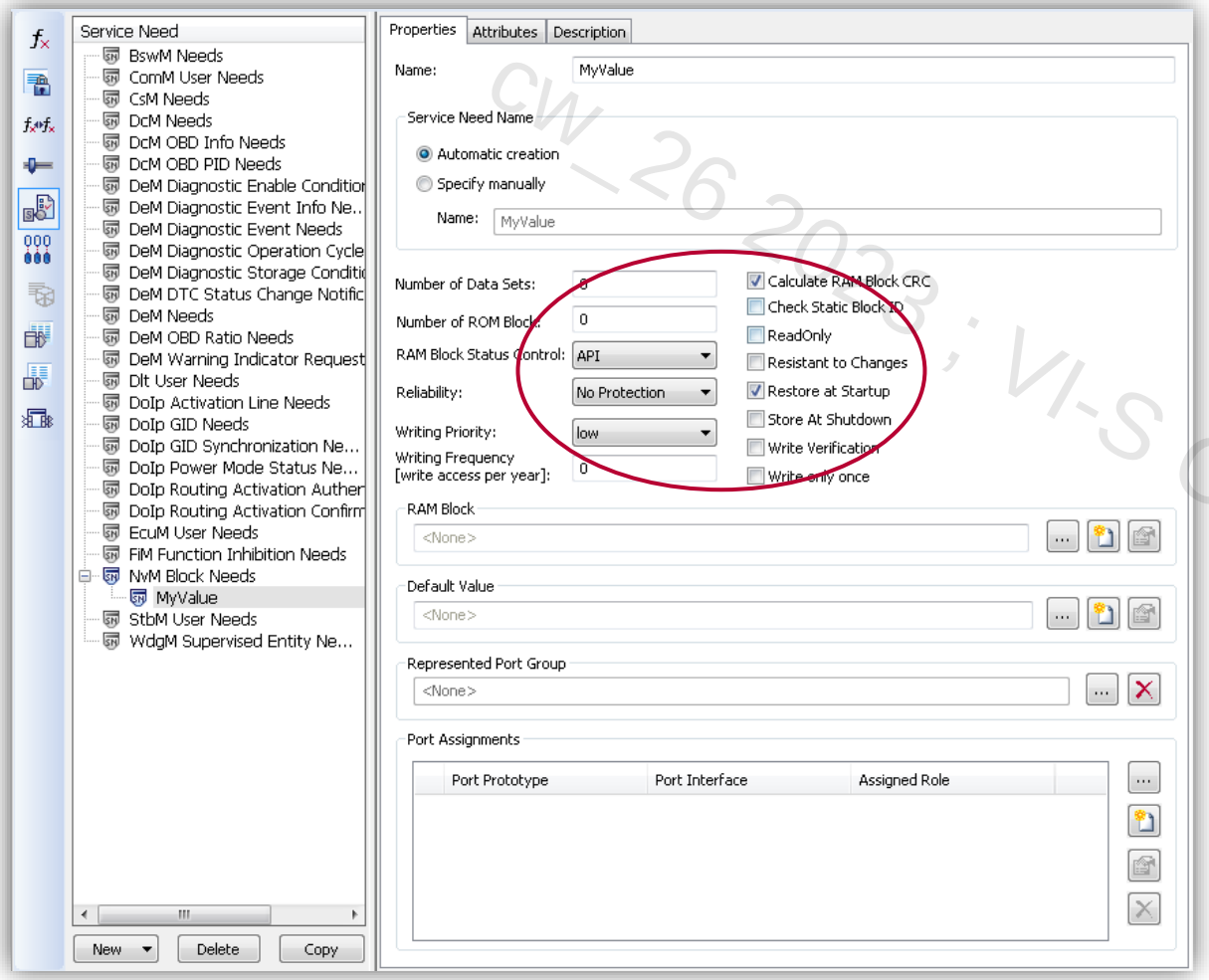
- DEV: Create a new NvM Block Service Need



CW_26_2023 ; VI-S CEA Seitech

Top-down service configuration (NvM)

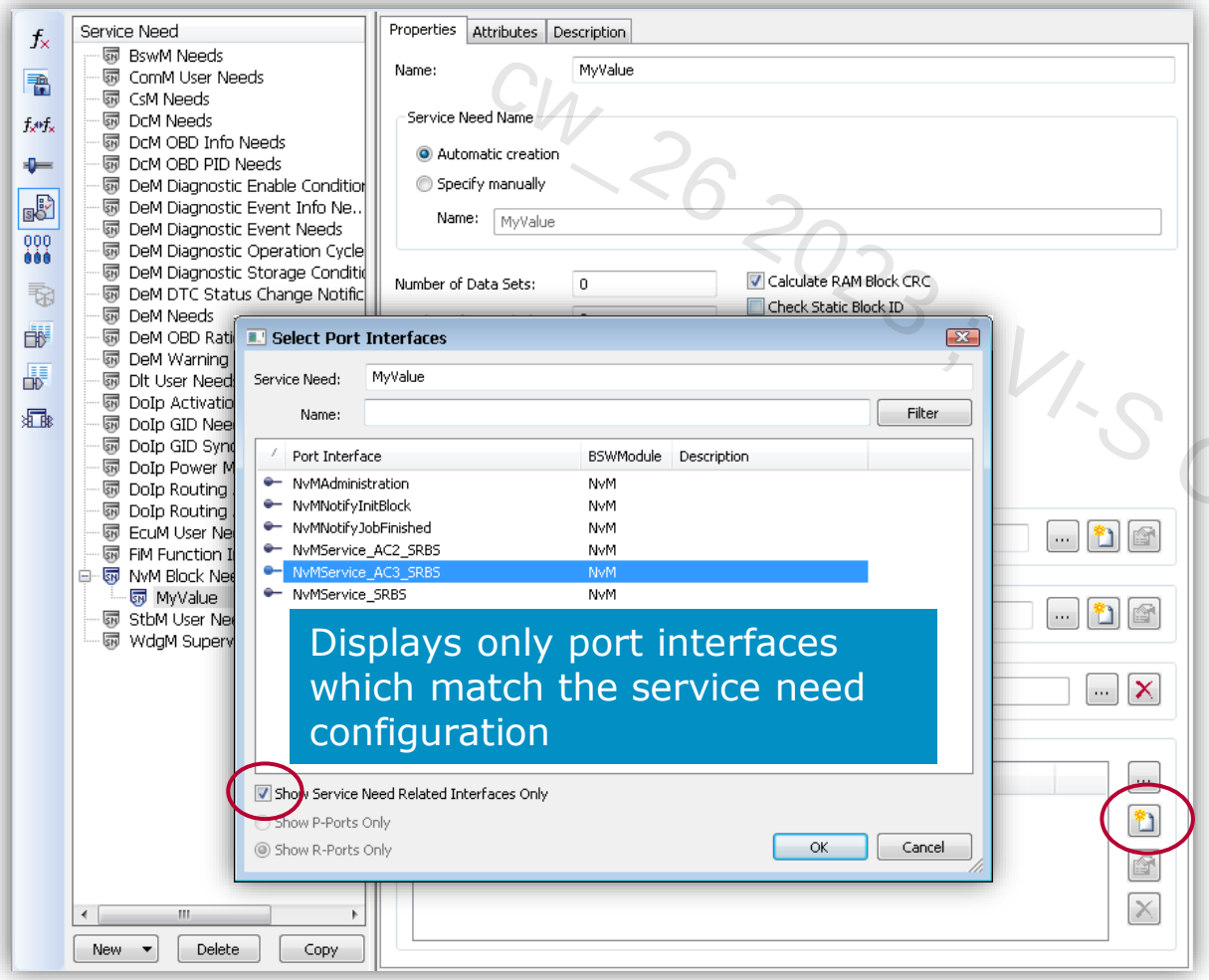
- DEV: Configure the service need properties



CW_26_2023 ; VI-S CEA Seitech

Top-down service configuration (NvM)

- DEV: Create service ports matching to the service need

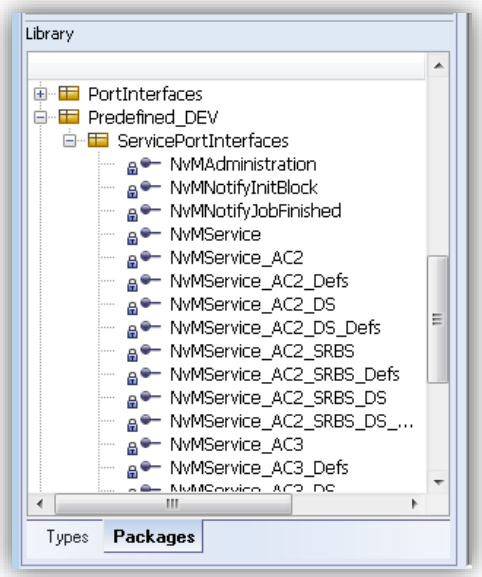


CW_26 2023, VI-S CEA Seitech

Top-down service configuration (NvM)

- Finding suitable service ports can be done top-down without NVM being configured
- DEV: **Predefined Service Port Interfaces**
  - File *ServicePortInterfaces_4.1.2.arxml* in tool installation directory
  - New workspace: automatically imported
  - Existing workspace: manual import required

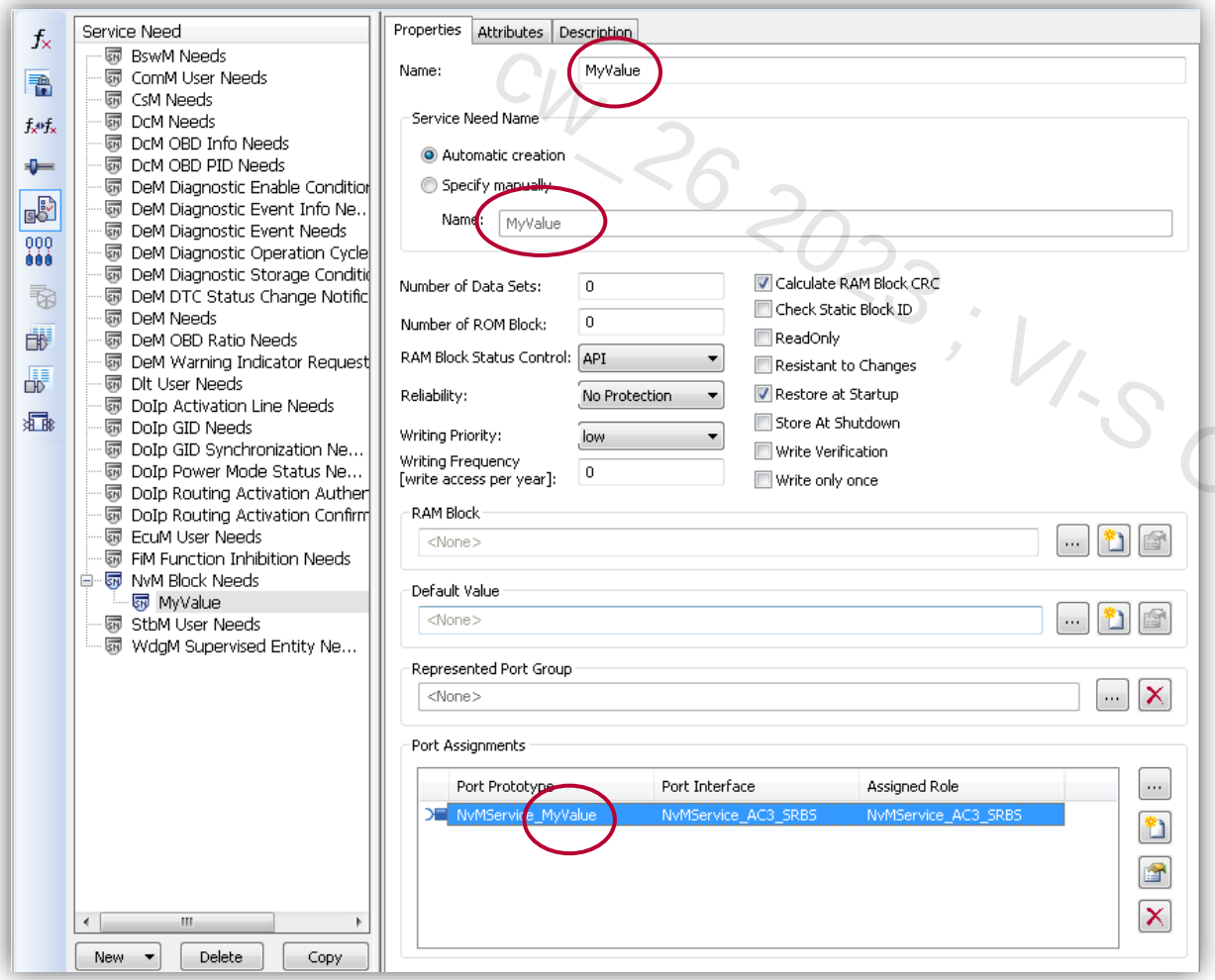
CW_26_2023 ; VI-S CEA Seitech



Developer: Predefined Service Port Interfaces after Import

Top-down service configuration (NvM)

- DEV: Create service ports matching the service need



This name is used for port creation

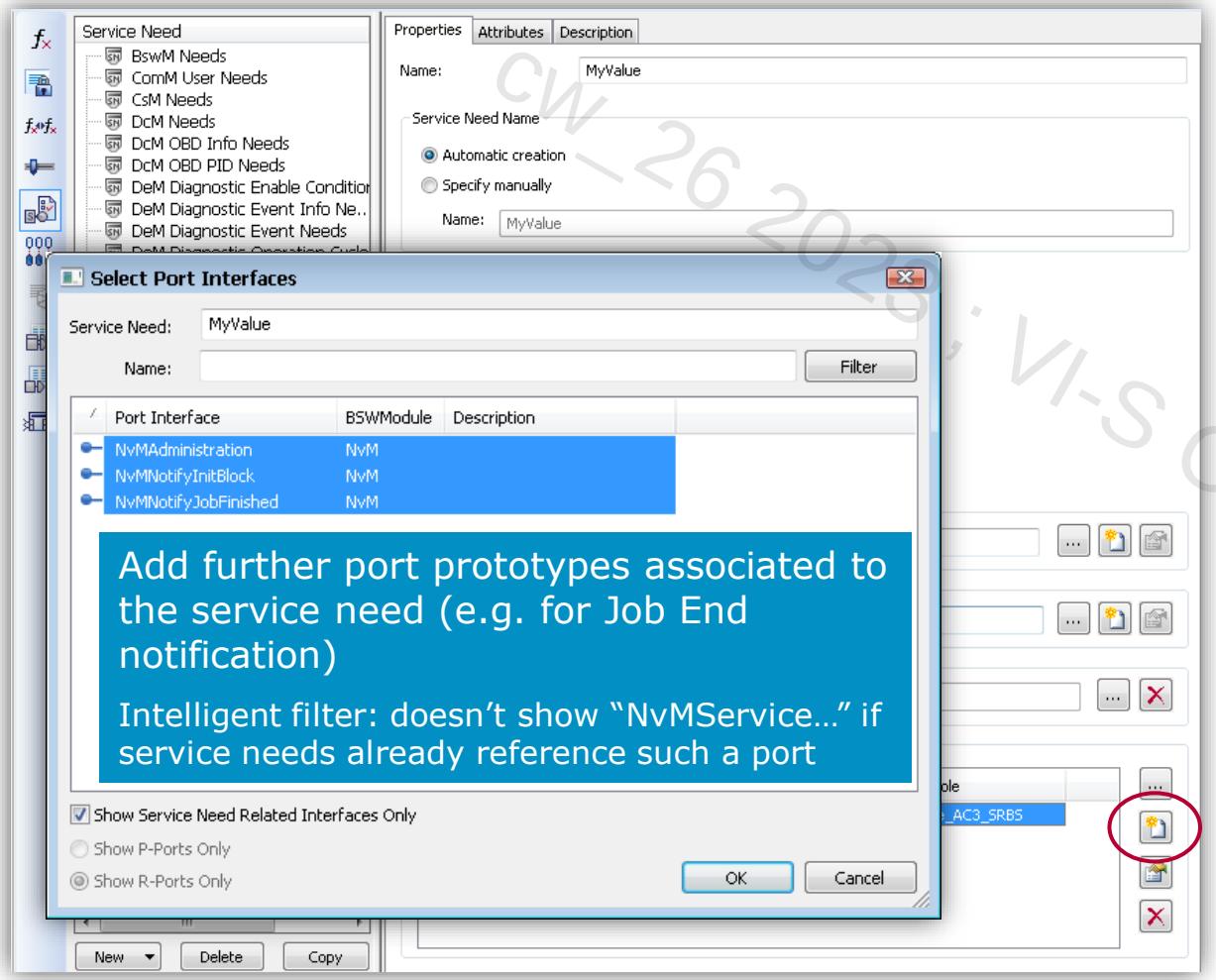
This name is used for EcuC config creation

Name of port automatically set

CW_26_2023 ; VI-S CEA Seitech

Top-down service configuration (NvM)

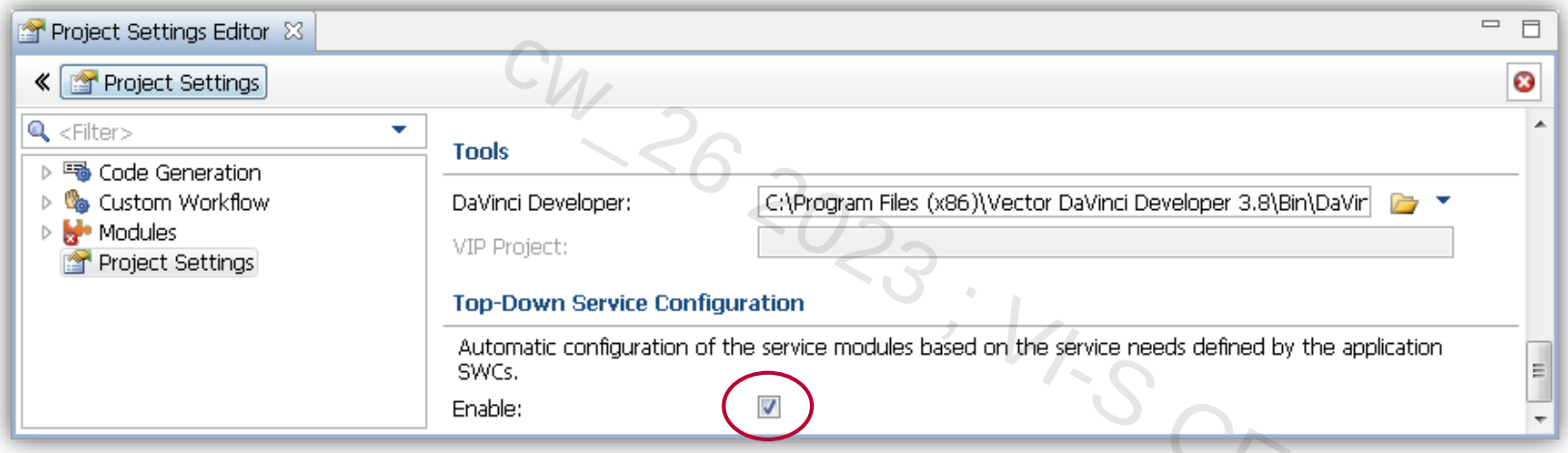
- DEV: Create service ports matching the service need



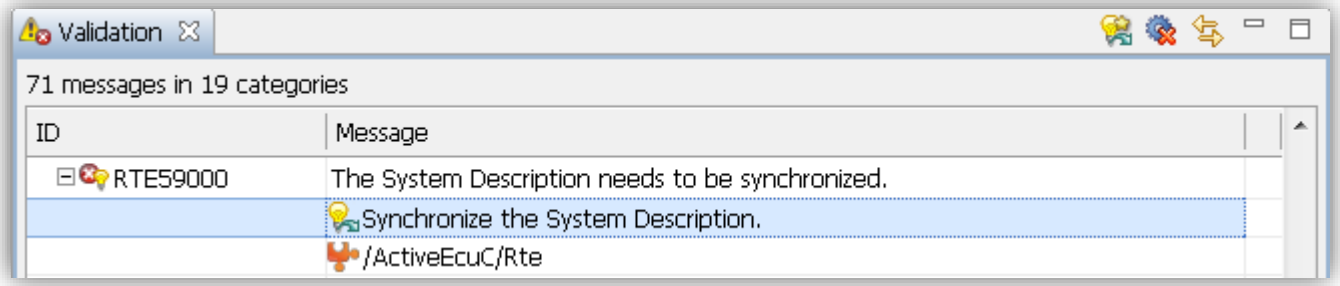
CW_26 2026 ; VI-S CEA Seitech

Top-down service configuration (NvM)

- CFG5: Enable top-down service configuration



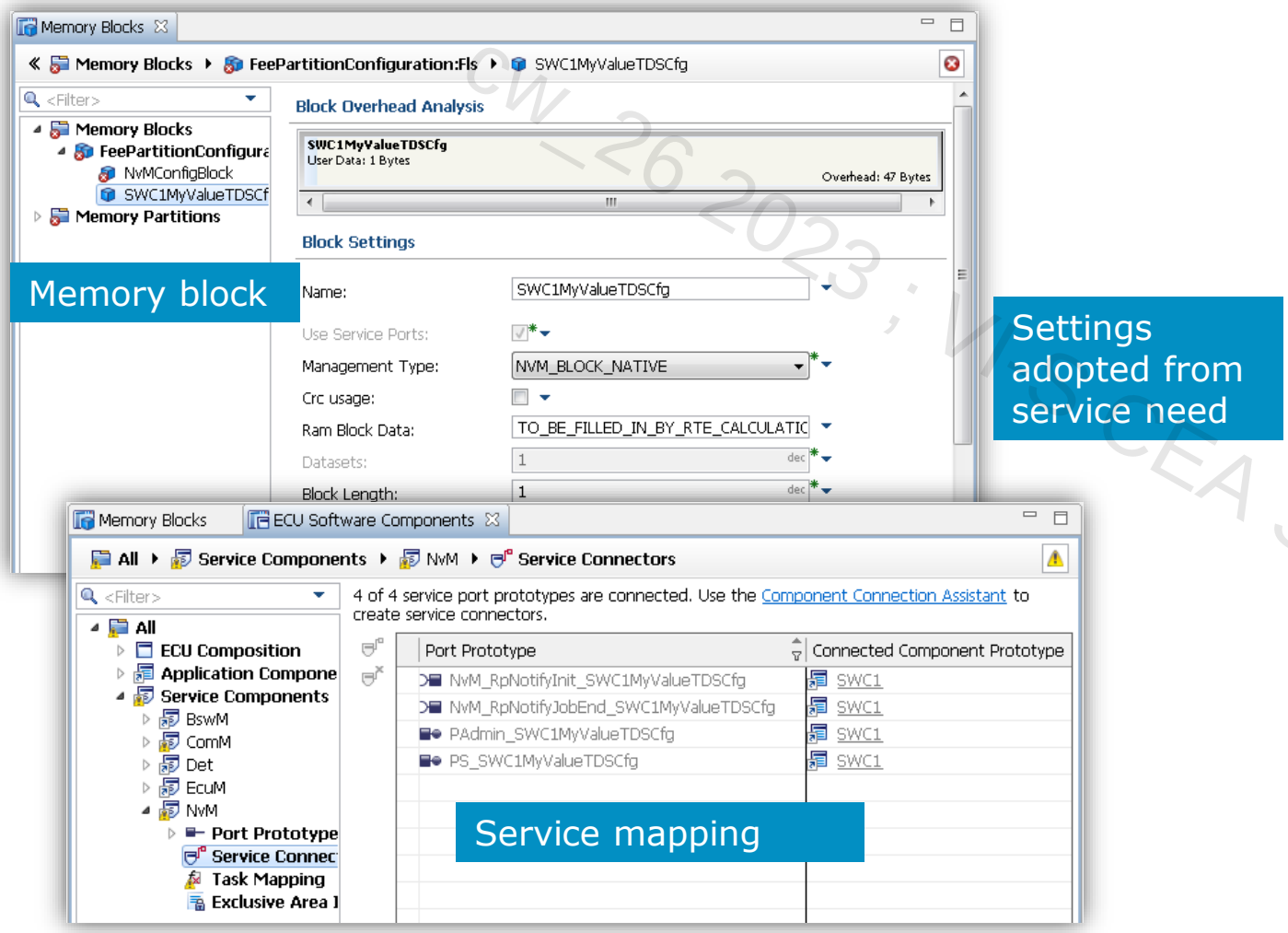
- CFG5: Run System Description synchronization



CW_26_2023 ; VI-S CEA Seitech

Top-down service configuration (NvM)

- CFG5: Result



cw_26_2023 ; VECTOR CEA Seitech

Top-down service configuration (NvM)

- DEV: Consistency check of ECU global NvM settings
  - Ensure same setting for “RAM Block Status Control”
  - Ensure that all “NvMService” ports have the same NvM API config class

cw_26 2023 ; VI-S CEA Seitech

Memory Abstraction

Diagnostics

cw_26 2023, VI-S CEA Seitech